

CS330 Operating Systems and Lab.

File Systems

Spring 2026

KAIST

Instructor : Insik Shin

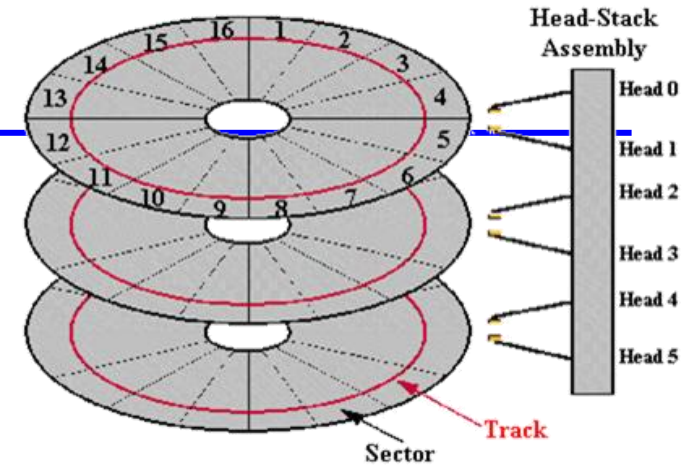
Course Outline

- Process & Threads
 - CPU scheduling, synchronization
- Memory Management
- File Systems & I/O

Hardware Resource	OS Abstraction
Machine	Processes / Threads
Memory	Address Space
Storage (Disk)	File

File Concept

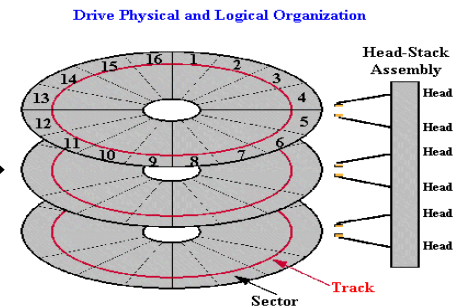
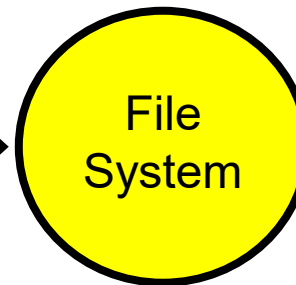
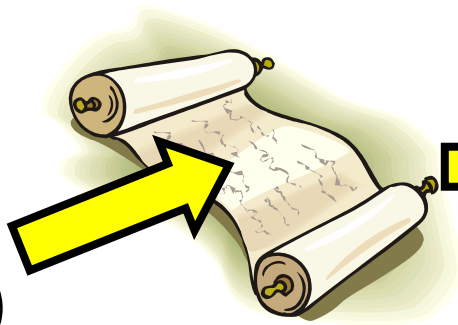
- What is it?



- Question: how does the user ask for a particular data?
 - One option: user specifies a sector number
 - Imagine: `read("14553344")` ➡ `read("abc.txt")`
 - Better option: specify by textual name
 - Have to map name → sector number
 - Another option: Icon
 - This is how Apple made its money. Graphical user interfaces. Point to a file and click

File Concept

- File abstraction
 - A logical storage unit
 - The smallest allotment for storing information on secondary storage, from a user's perspective
 - A named collection of related information



- User programs
 - File

- Devices (Disk)
 - Block (Sector)

File Concept

- File concept
 - User's view - named sequence of bytes
 - FS's view - collection of disk blocks
- File system's job
 - Translate name & offset to disk blocks

File Systems

- File systems
 - Resides on secondary storage (disks)
 - Persistent (non-volatile): contents don't go away
 - Provides efficient and convenient access to storage (disk) by allowing data to be stored, located, retrieved easily
- File attributes & operations

File Operations

- File is an **abstract data type**
- File operations:
 - **Create**
 - **Write**
 - **Read**
 - **Reposition within file**
 - **Delete**
 - **Truncate**

File Attributes

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk

Directory

- Suppose you want to visit my office.
 - You may wonder where my office is:



Directory

- Suppose you get into E3-1.
 - You may wonder which office is mine
 - Then, you can look up a **directory** located in the first floor entrance



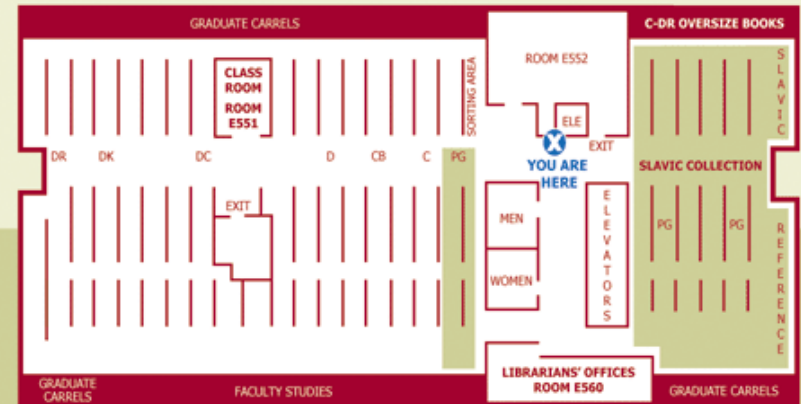
Office: E3-1, 4425

Directory



Research Collections (East Tower) Directory

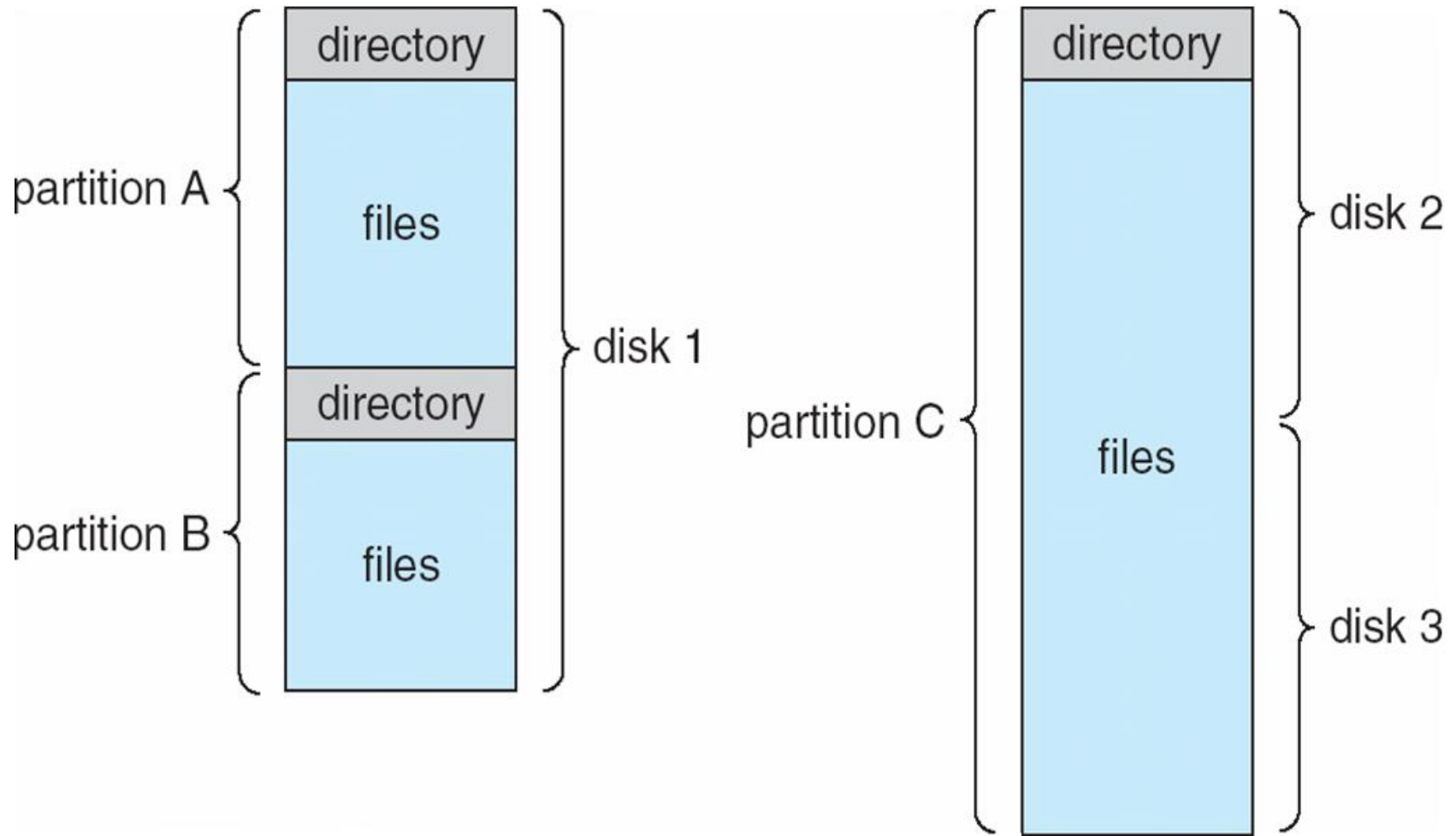
5TH FLOOR: C-DR & PG



- > **Slavic Collection**
- > **C-DR Oversize Books**
- < **Classroom E551**
- v **Librarians' Offices E560**

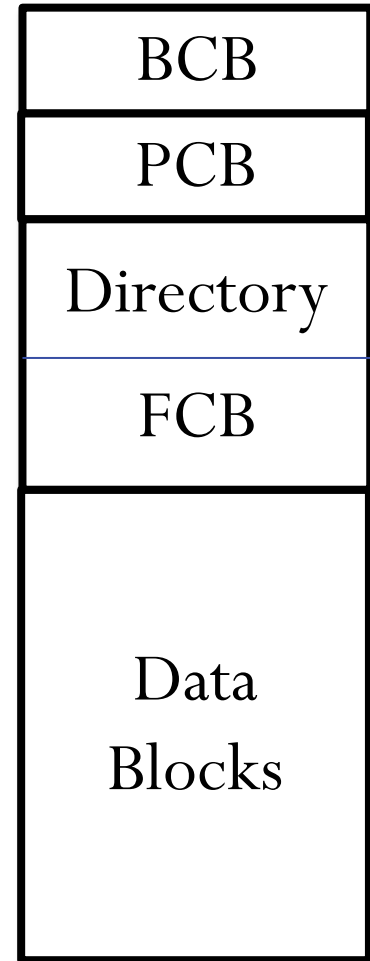
Research Collections	Floor	Special Collections	Floor
AC-BX	4	African Studies (DTs)	6
C-DR	5	East Asian (Or. Ref., O.C., O.J., O.K.)	8
DS-GN	6	Folklore	7
GR-HX	7	Government Information, Microforms, Statistics	2
J-NX	8	IU Dissertations (North end of the floor)	6
P-PF	9	IU Reference Collection	1
PG	5	Latin American Studies	6
PH-PQ7080	9	Reference Reading Room	1
PQ7081-9999	6	Secure Area	Ask at Circ. Desk
PR-SF	10	Slavic (PGs)	5
SH-Z	11	Tibetan	9

A Typical File-system Organization



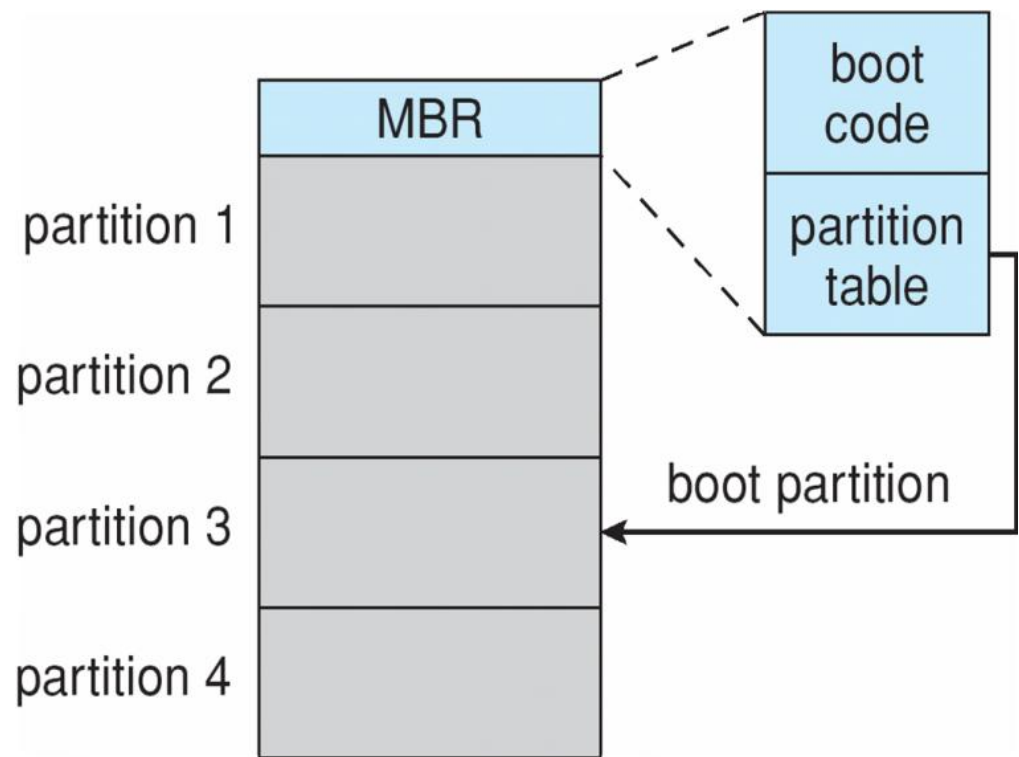
File-System Implementation

- **Boot Control Block (BCB)** contains info needed by system to boot OS from that partition
 - Needed if partition contains OS, usually first block of partition
- **Partition Control Block (PCB)** contains partition details (FS metadata)
 - Total # of blocks, # of free blocks, block size, free block pointers
- **Directory structure** organizes the files
 - Names and inode numbers, master file table
- Per-file **File Control Block (FCB)** contains many details about the file (file metadata)
 - Inode number, permissions, size, dates



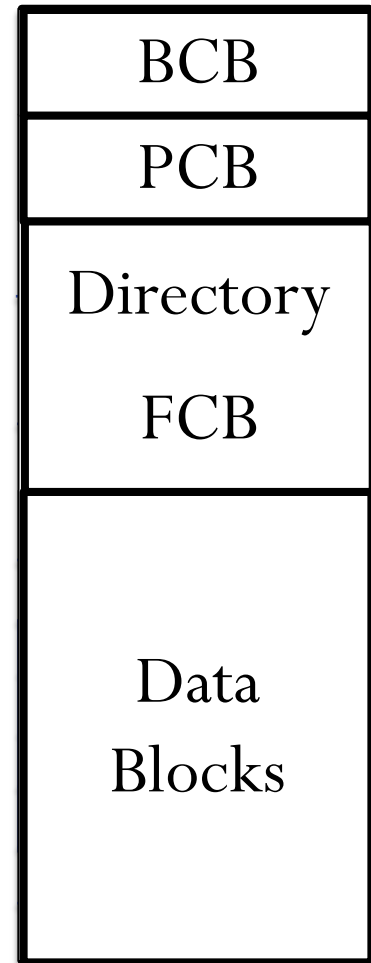
Disk Management

- Bootstrap program
 - Initializes all aspects of the system, from CPU registers to device controllers, main memory, and starts the operating system
 - **Bootstrap loader** stored in ROM & in boot disk
 - Operating systems



File-System Implementation

- **Boot Control Block (BCB)** contains info needed by system to boot OS from that partition
 - Needed if partition contains OS, usually first block of partition
- **Partition Control Block (PCB)** contains partition details (FS metadata)
 - Total # of blocks, # of free blocks, block size, free block pointers
- **Directory structure** organizes the files
 - Names and inode numbers, master file table
- Per-file **File Control Block (FCB)** contains many details about the file (file metadata)
 - Inode number, permissions, size, dates



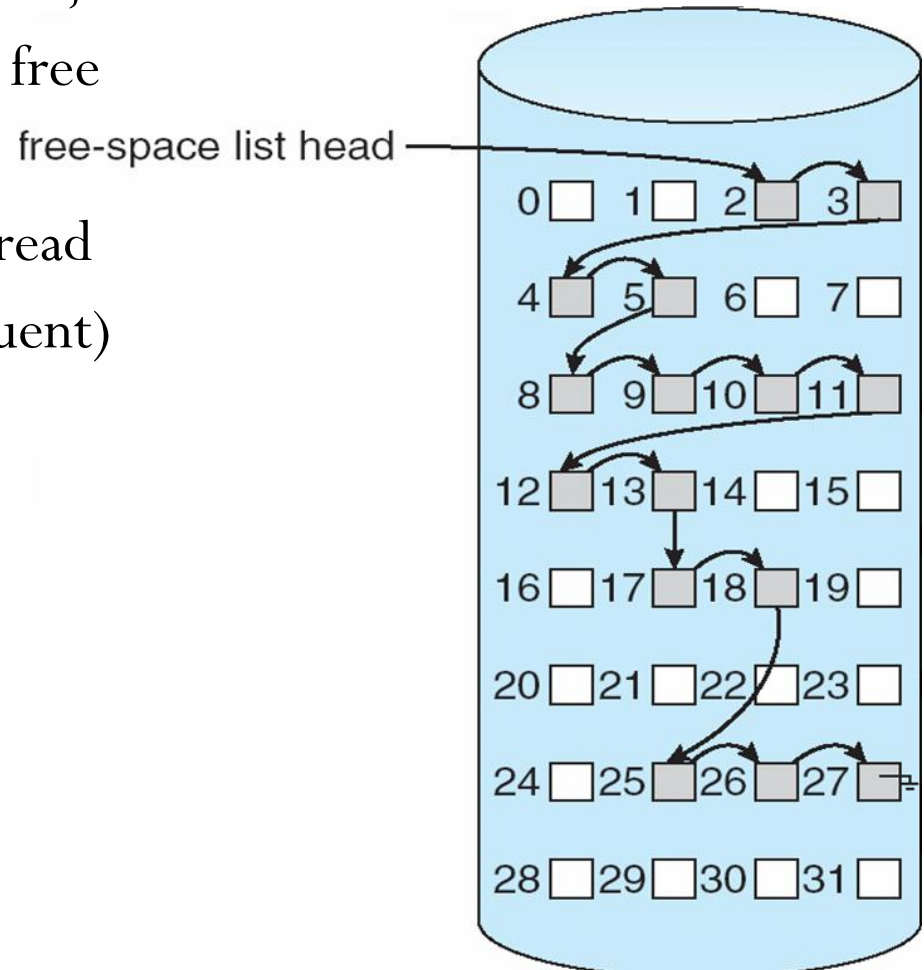
Free-Space Management

- Bitmap or bit vector
 - Each block is represented by a single-digit bit
 - 1 = free, 0 = allocated
 - Simple and efficient in finding the first free block, or n consecutive free blocks
 - May be accelerated by CPU's bit-manipulation instructions
 - Inefficient unless the entire vector is kept in main memory

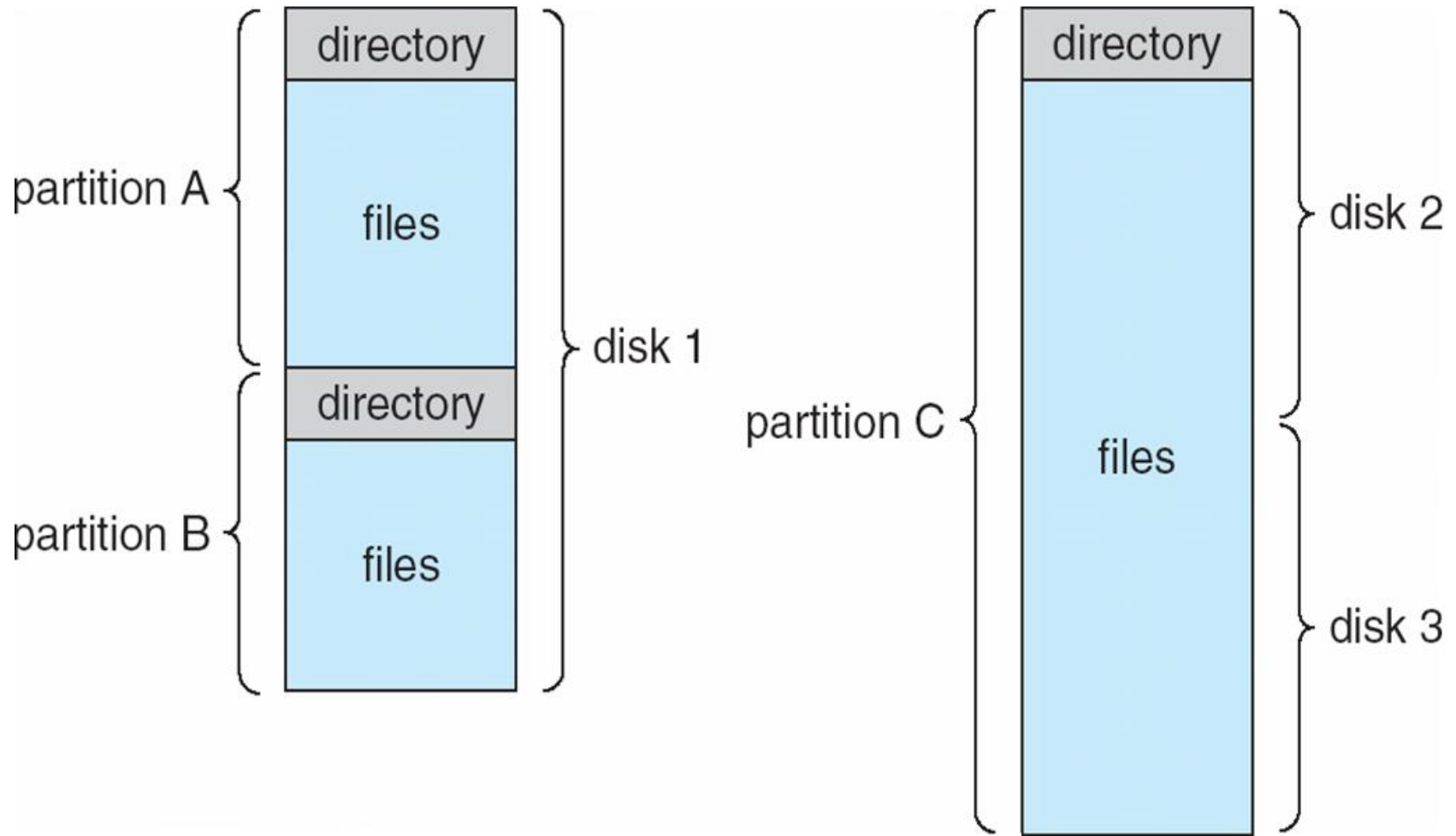
	0	1	2	3	4	5	6	7
Free blocks	0	0	1	0	1	0	0	0

Free-Space Management

- Linked list
 - Link together all the free blocks, keeping a pointer to the first free block
 - To traverse the list, we must read each block (but it is not frequent)

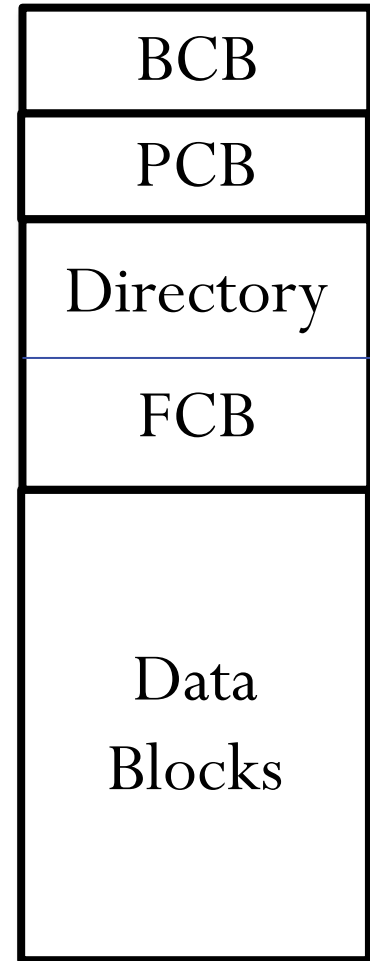


A Typical File-system Organization



File-System Implementation

- **Boot Control Block (BCB)** contains info needed by system to boot OS from that partition
 - Needed if partition contains OS, usually first block of partition
- **Partition Control Block (PCB)** contains partition details (FS metadata)
 - Total # of blocks, # of free blocks, block size, free block pointers
- **Directory structure** organizes the files
 - Names and inode numbers, master file table
- Per-file **File Control Block (FCB)** contains many details about the file (file metadata)
 - Inode number, permissions, size, dates



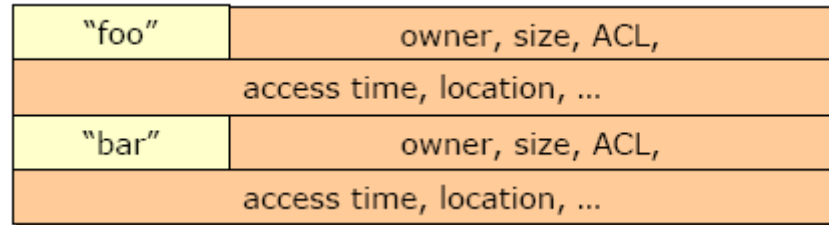
File Control Block (FCB)

- FCB contains many details about the file (file metadata)

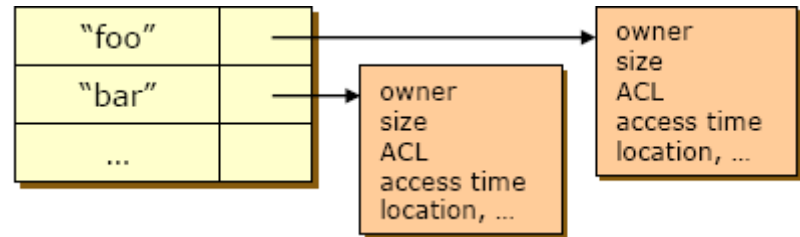
file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

Directory Structure

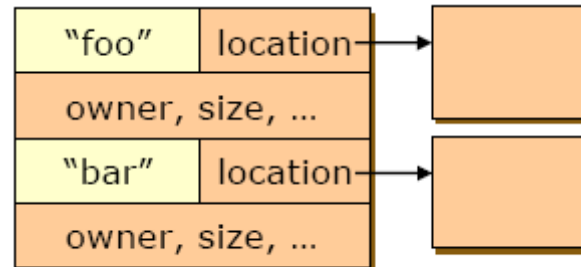
- Directory implementation
 - Table (fixed length)



- Linear list of file names with pointer to the data blocks
 - simple to program
 - time-consuming to execute

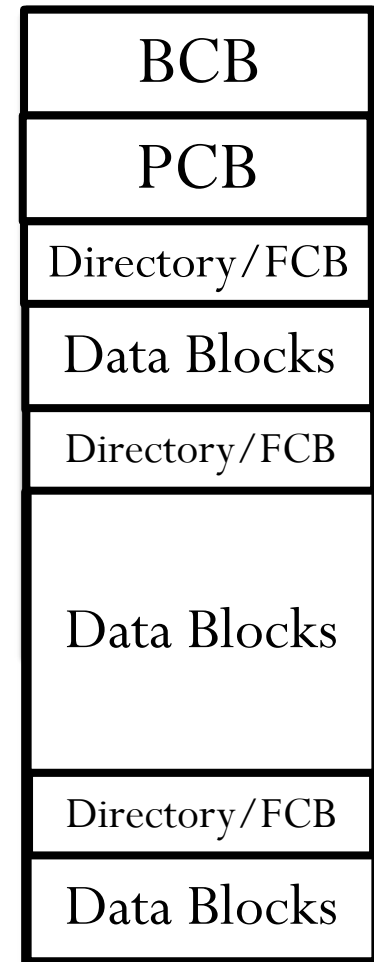


- Hybrid approach



File-System Implementation

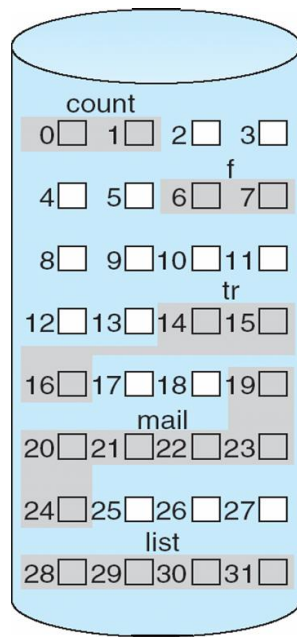
- **Boot Control Block (BCB)** contains info needed by system to boot OS from that partition
 - Needed if partition contains OS, usually first block of partition
- **Partition Control Block (PCB)** contains partition details (FS metadata)
 - Total # of blocks, # of free blocks, block size, free block pointers
- **Directory structure** organizes the files
 - Names and inode numbers, master file table
- Per-file **File Control Block (FCB)** contains many details about the file (file metadata)
 - Inode number, permissions, size, dates



Allocation Methods

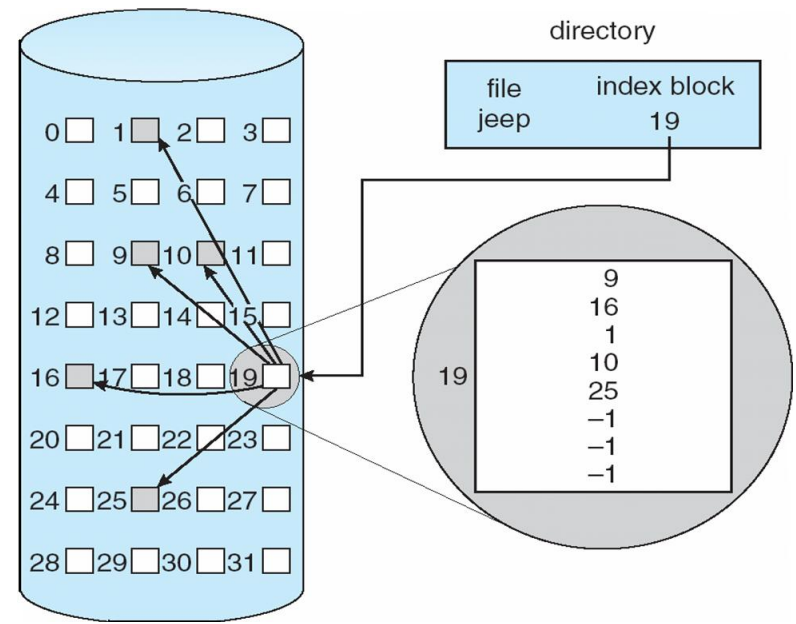
- Allocation methods for how to allocate disk blocks to files:

- Contiguous allocation



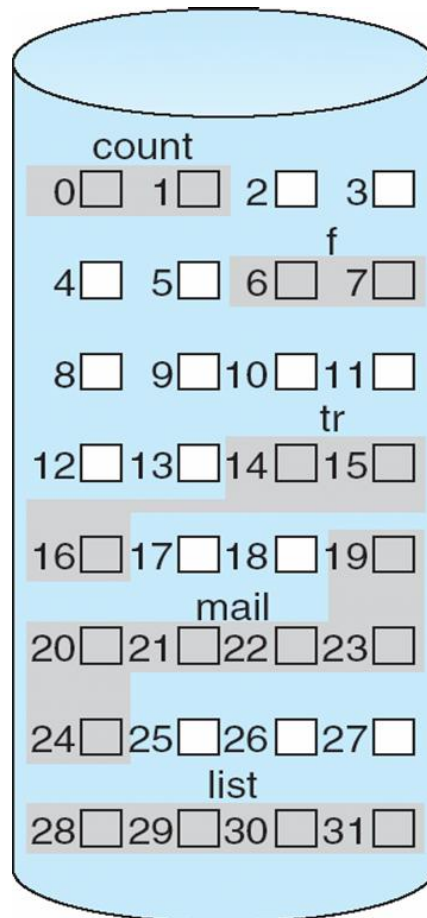
directory			
file	start	length	
count	0	2	
tr	14	3	
mail	19	6	
list	28	4	
f	6	2	

- Indexed allocation



Contiguous Allocation

- A file occupies a set of contiguous blocks on the disk



directory

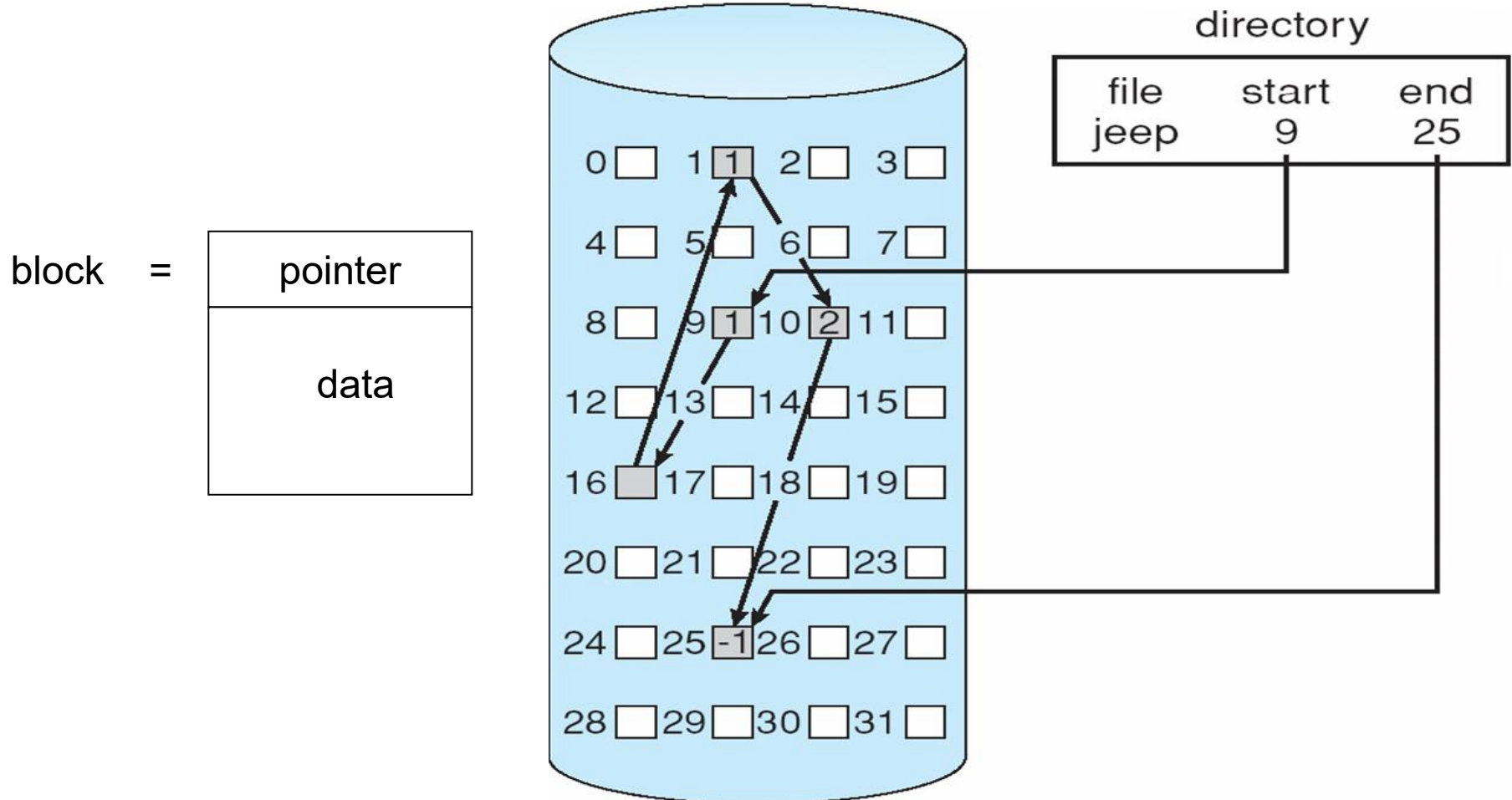
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Contiguous Allocation

- Advantages
 - Simple – only starting location (block #) and length (number of blocks) are required
 - Random access
 - Efficient access
 - The number of disk seeks is minimal
- Drawbacks
 - Wasteful of space (dynamic storage-allocation problem)
 - Less flexibility
 - Files cannot grow

Linked Allocation

- Each file is a linked list of disk blocks
 - blocks may be scattered anywhere on the disk.

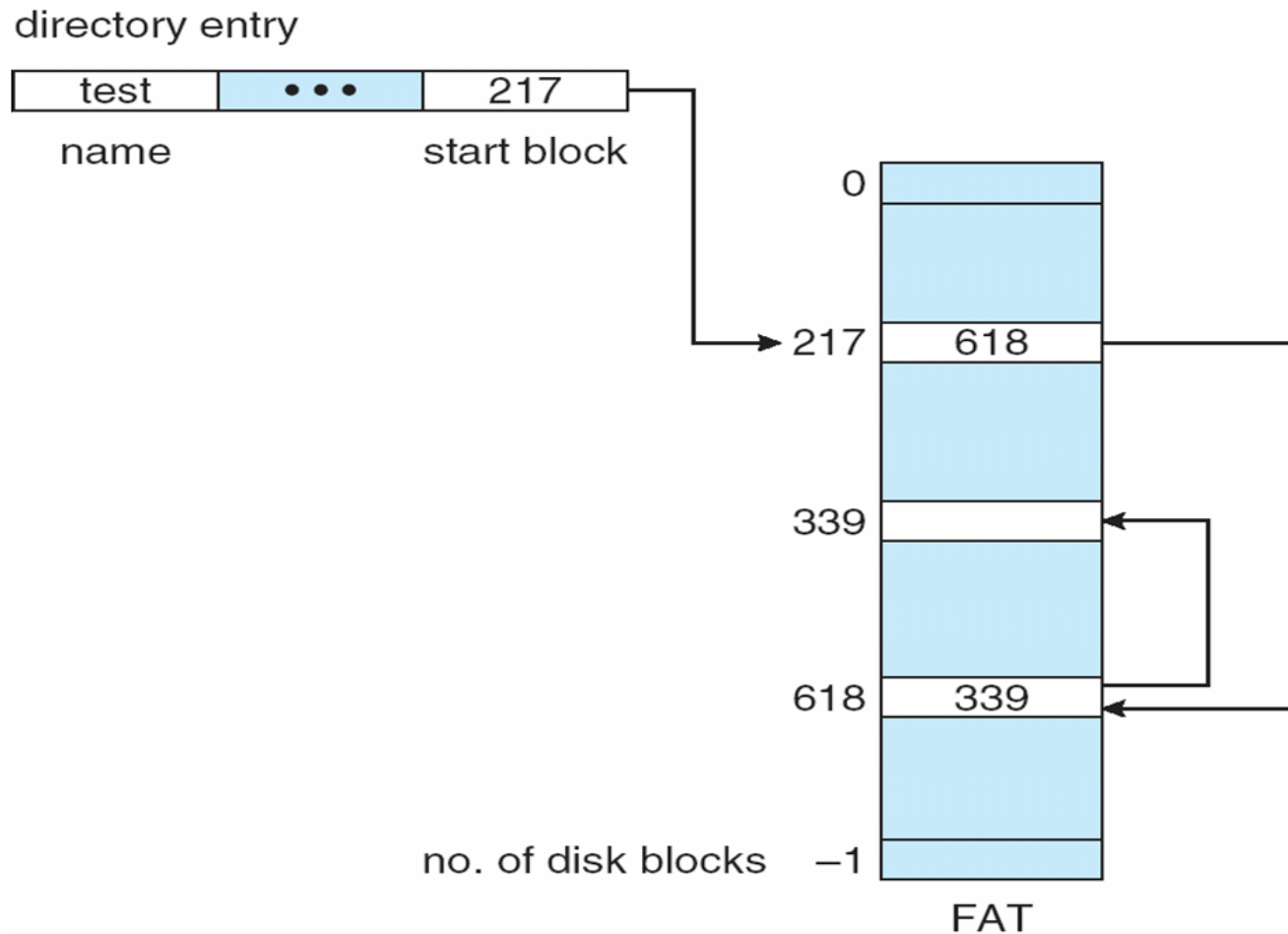


Linked Allocation (Cont.)

- Advantages
 - Simple – need only starting address
 - Free-space management system – no waste of space
- Disadvantage
 - No random access

File-Allocation Table

- File-allocation table (FAT)
 - disk-space allocation used by MS-DOS and OS/2.

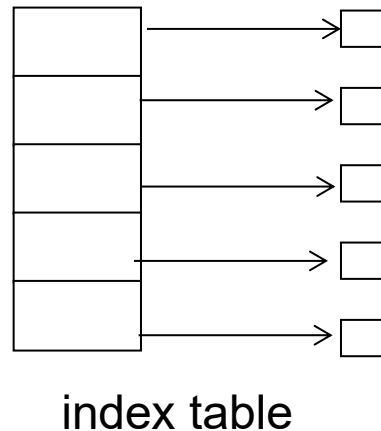


Indexed Allocation

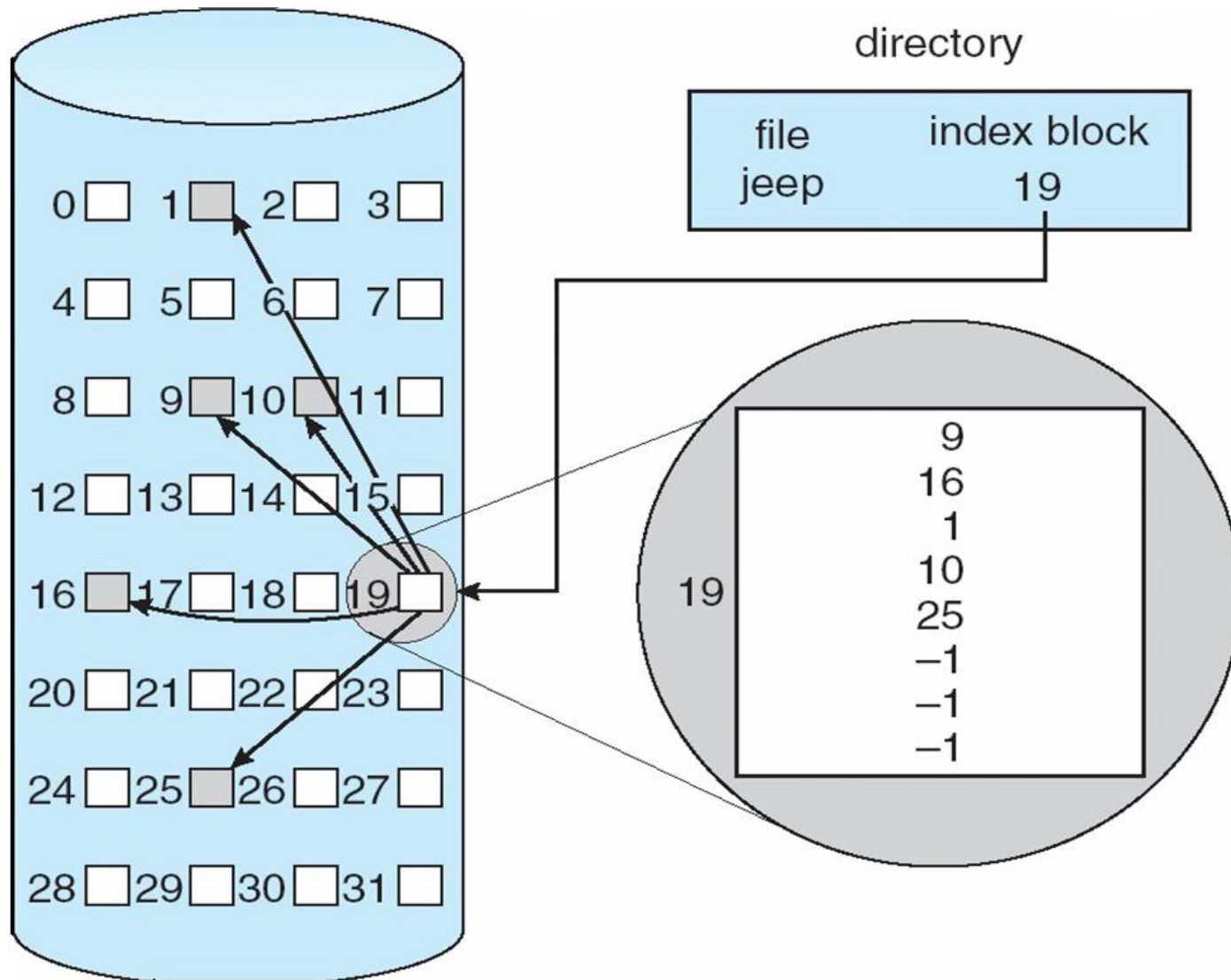
- **Indexed allocation**

- Each file has its own **index block**(s) of pointers to its data blocks
- It brings all pointers together into the **index block**

- Logical view



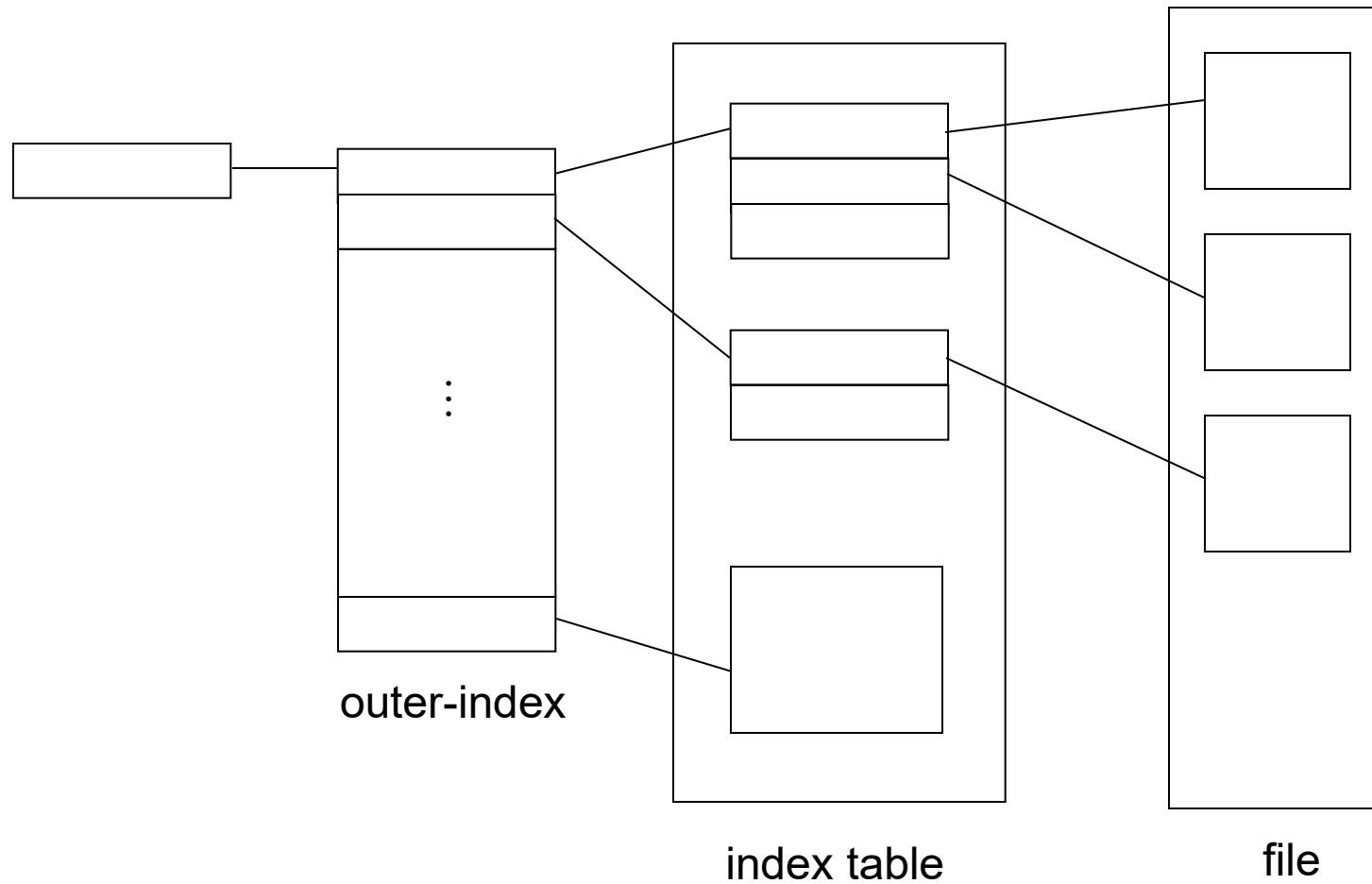
Example of Indexed Allocation



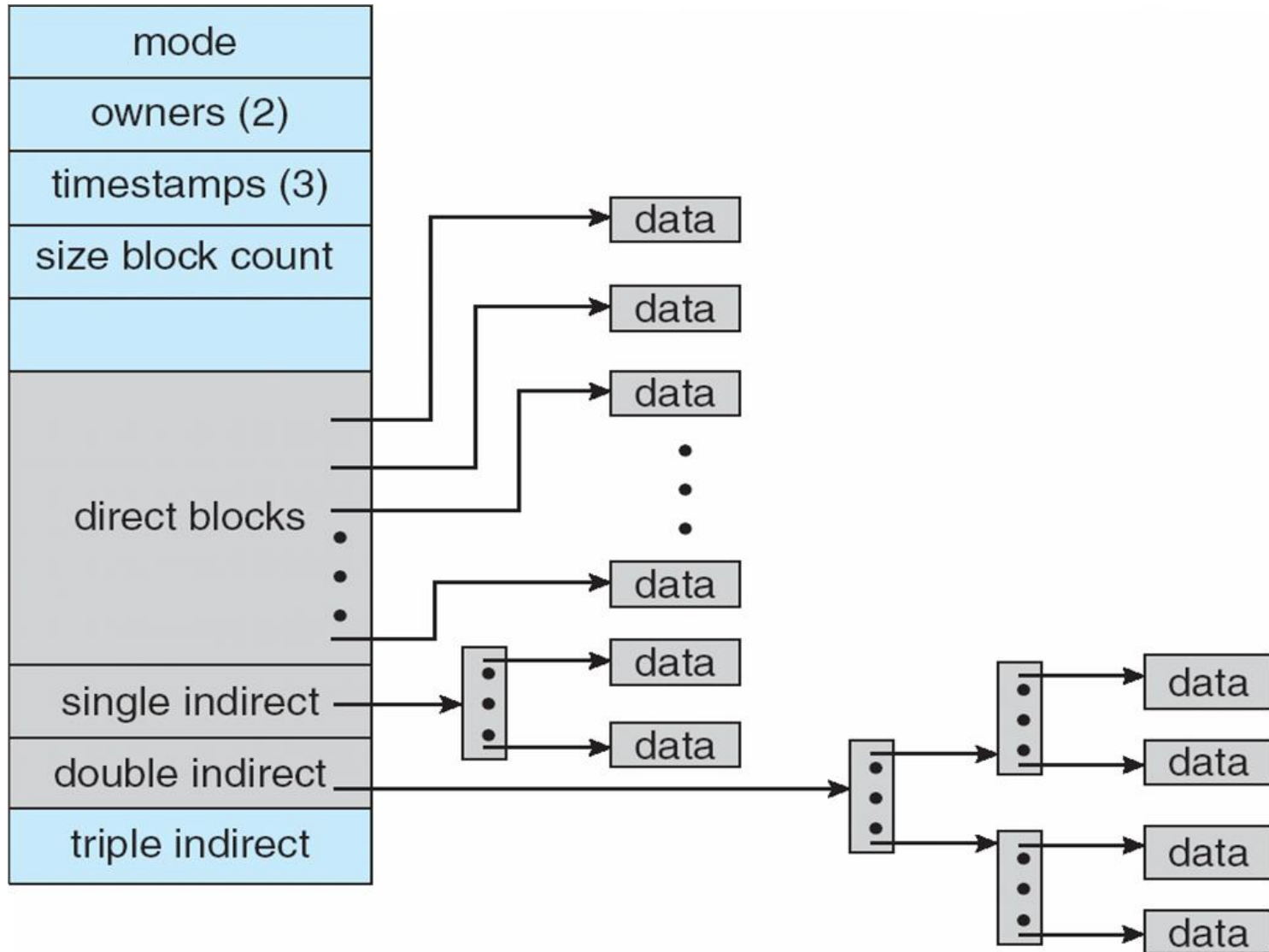
Indexed Allocation (Cont.)

- Need index table
- Random access
- Dynamic access without external fragmentation, but has overhead of index block

Indexed Allocation – Mapping (Cont.)

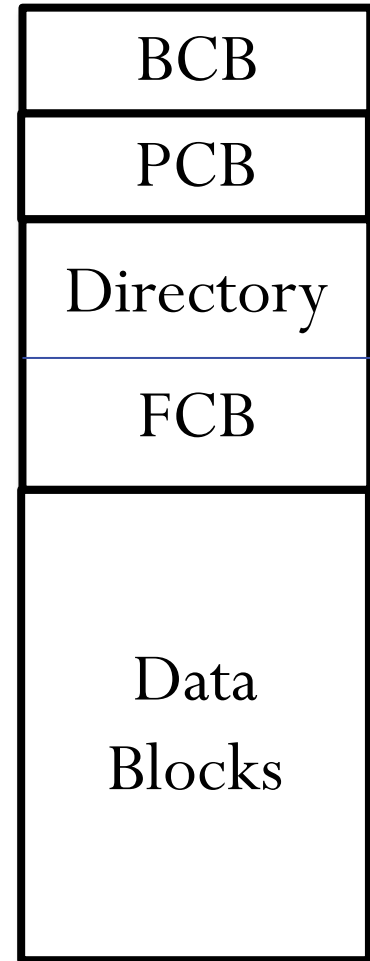


Combined Scheme: UNIX UFS (4K bytes per block)



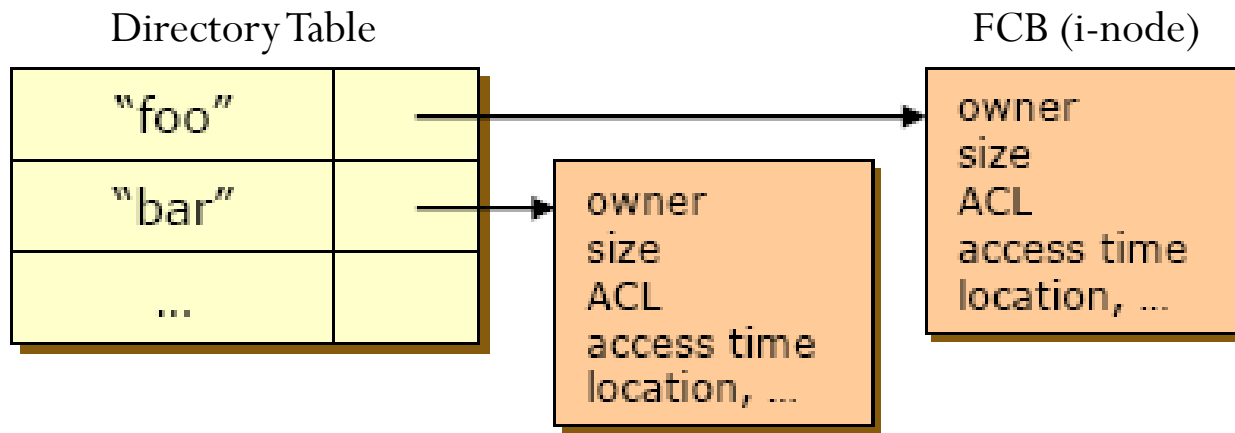
File-System Structure

- **Boot Control Block (BCB)** contains info needed by system to boot OS from that partition
- **Partition Control Block (PCB)** contains partition details (FS metadata)
- **Directory structure** organizes the files
 - Names and inode numbers, master file table
- Per-file **File Control Block (FCB)** contains many details about the file (file metadata)
 - Inode number, permissions, size, dates



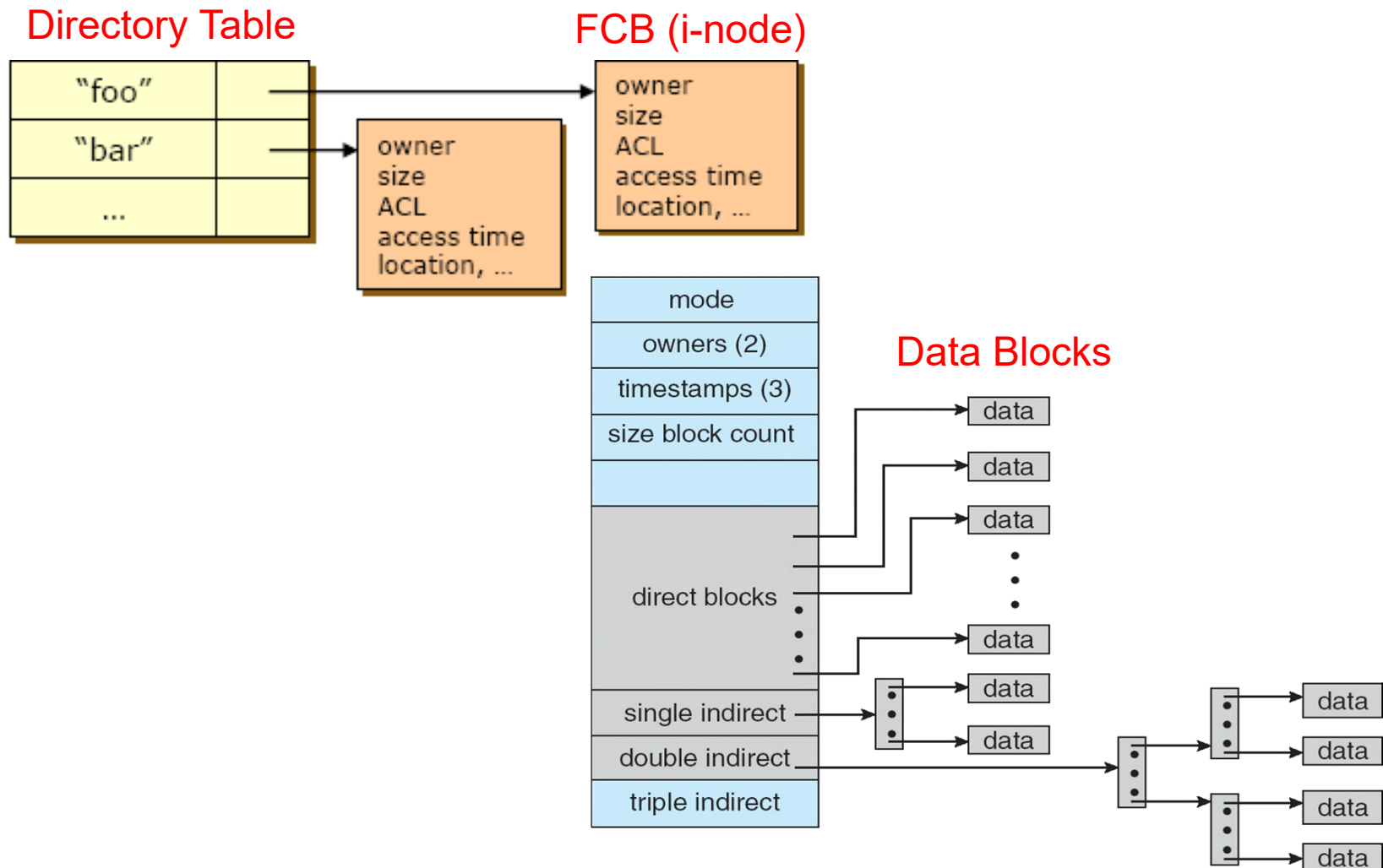
Directory Structure

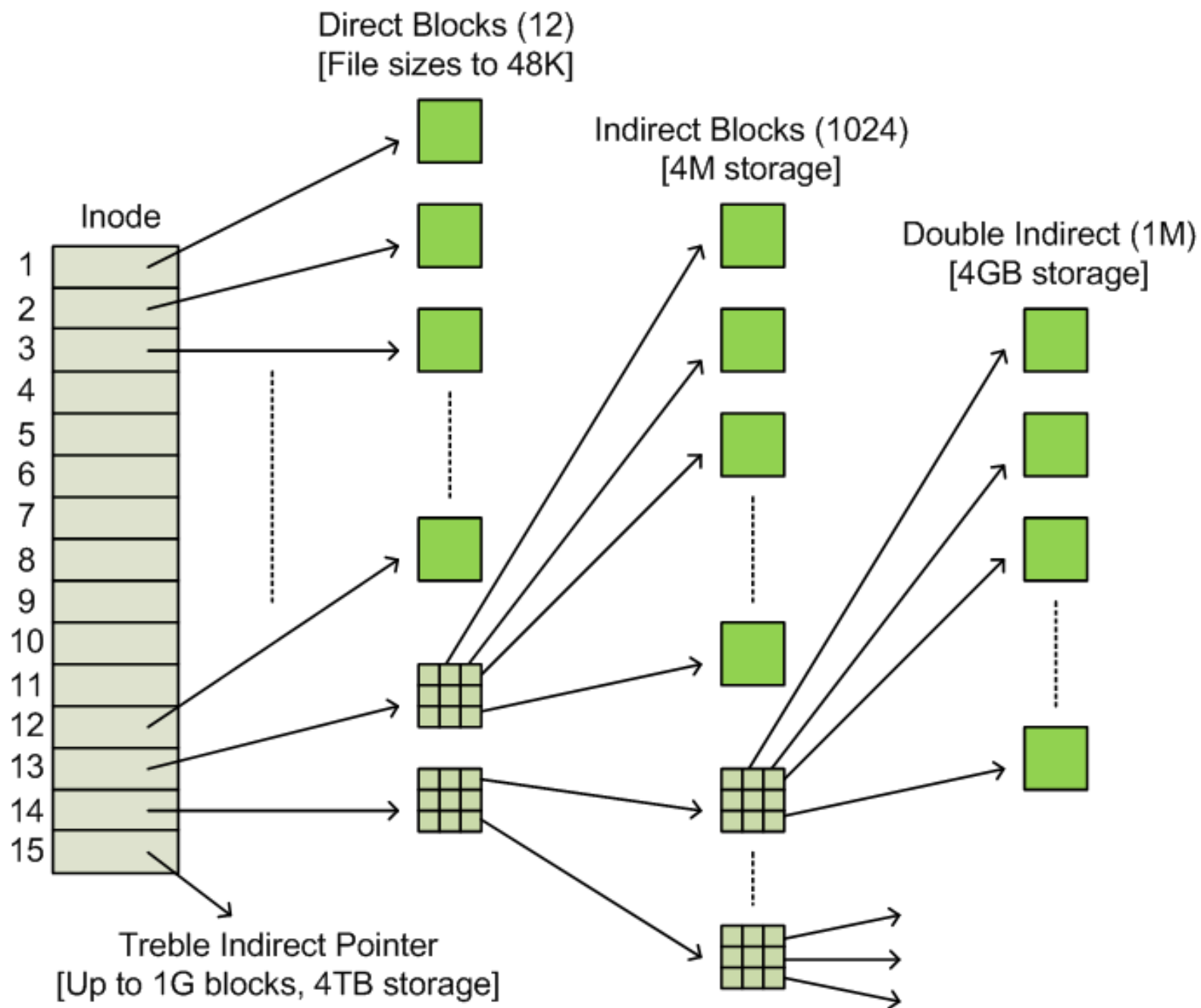
- Metadata are stored in the file control block (i.e., i-node)



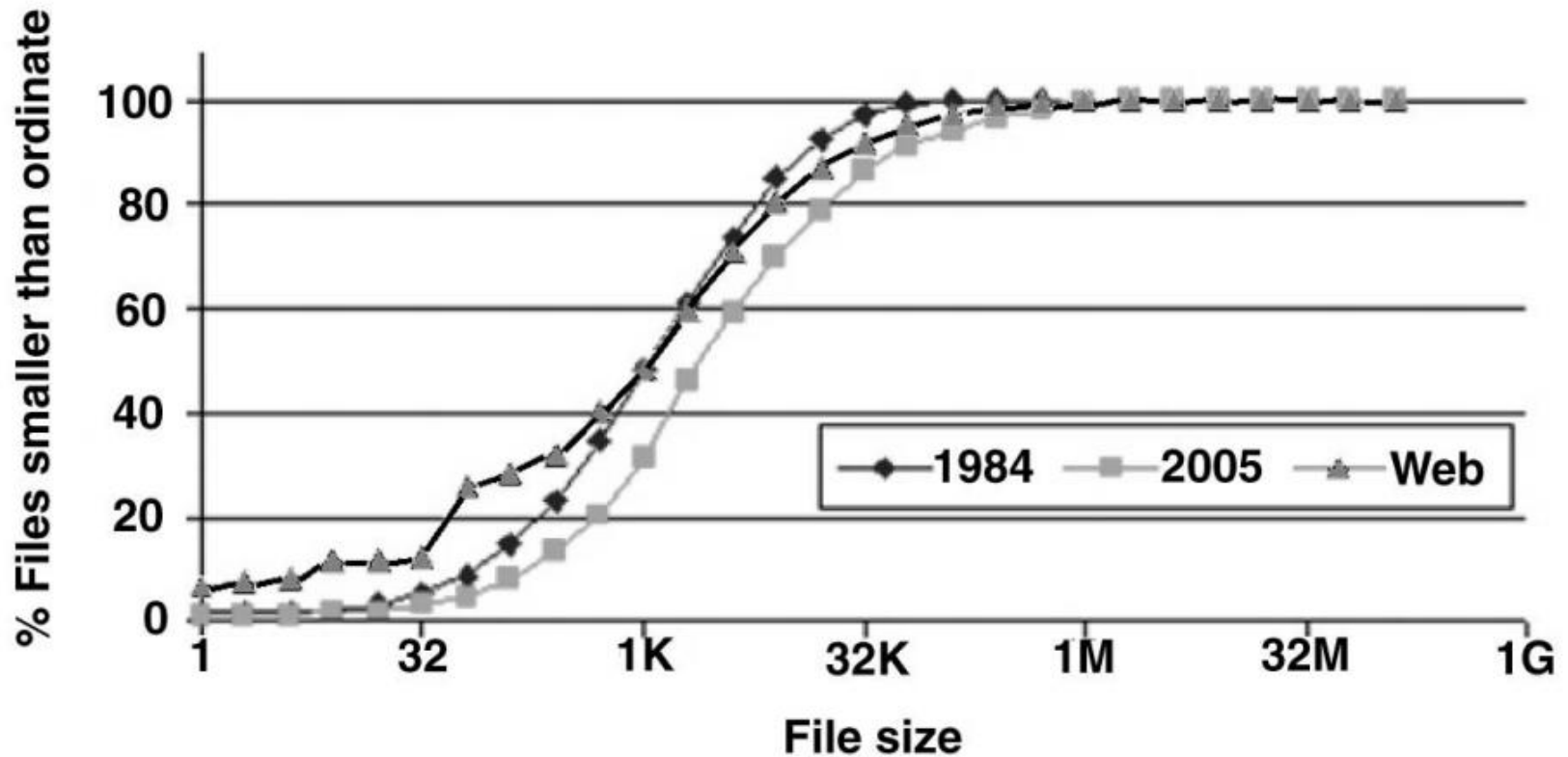
Directory Structure

- Metadata are stored in the file control block (i.e., i-node)





File Size Distribution



File Types

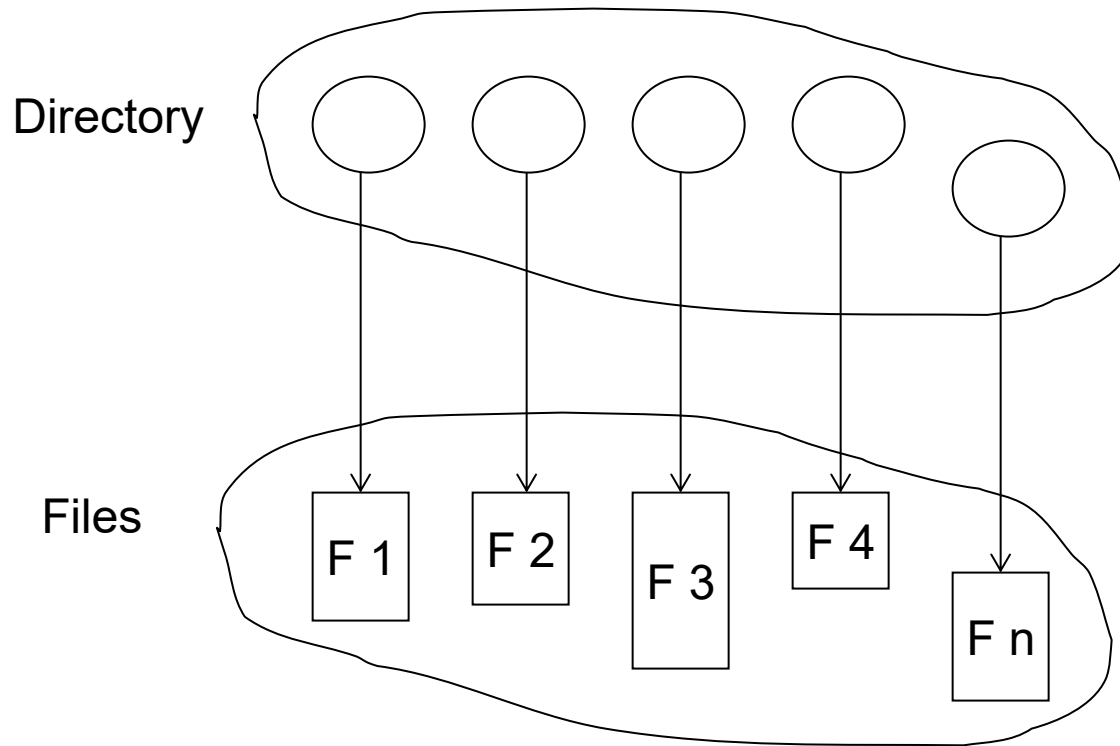
- File types help OS to perform reasonable operations on files, i.e.,
 - What kinds of operations can OS do?
 - ◆ Execute appropriate applications when double-clicking the mouse on some file icons
 - ◆ Disallow printing binary executable programs

File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

Directory Structure

- A collection of nodes containing information about all files

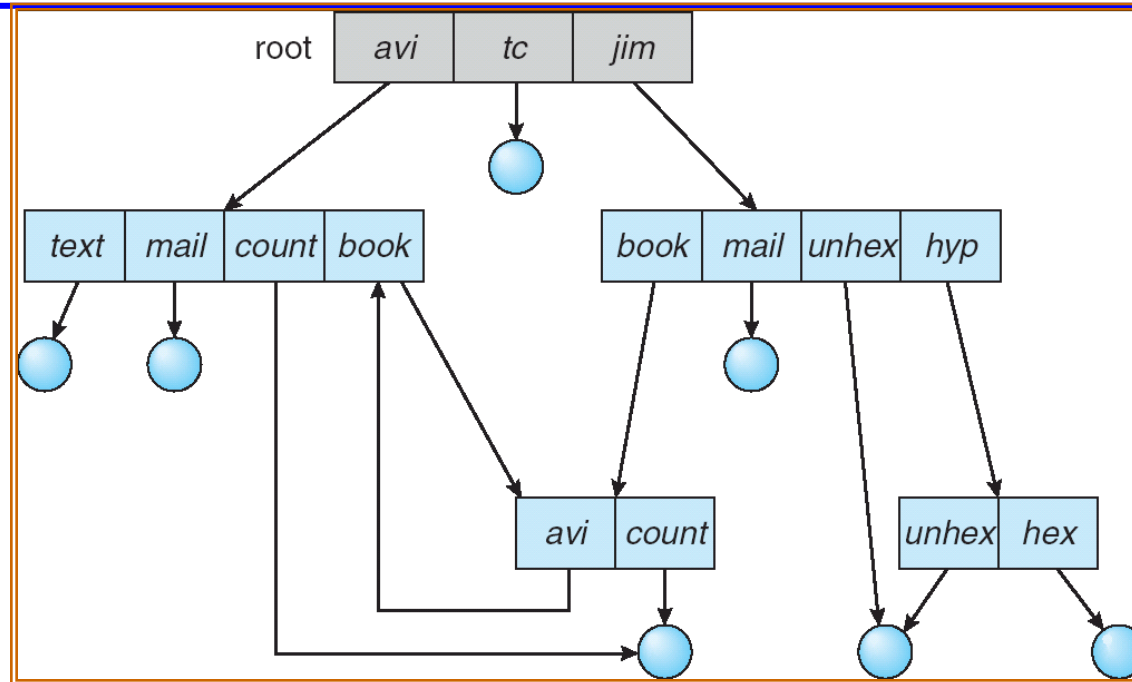


Both the directory structure and the files reside on disk
Good design principle: a directory itself as a file

Directory Organization

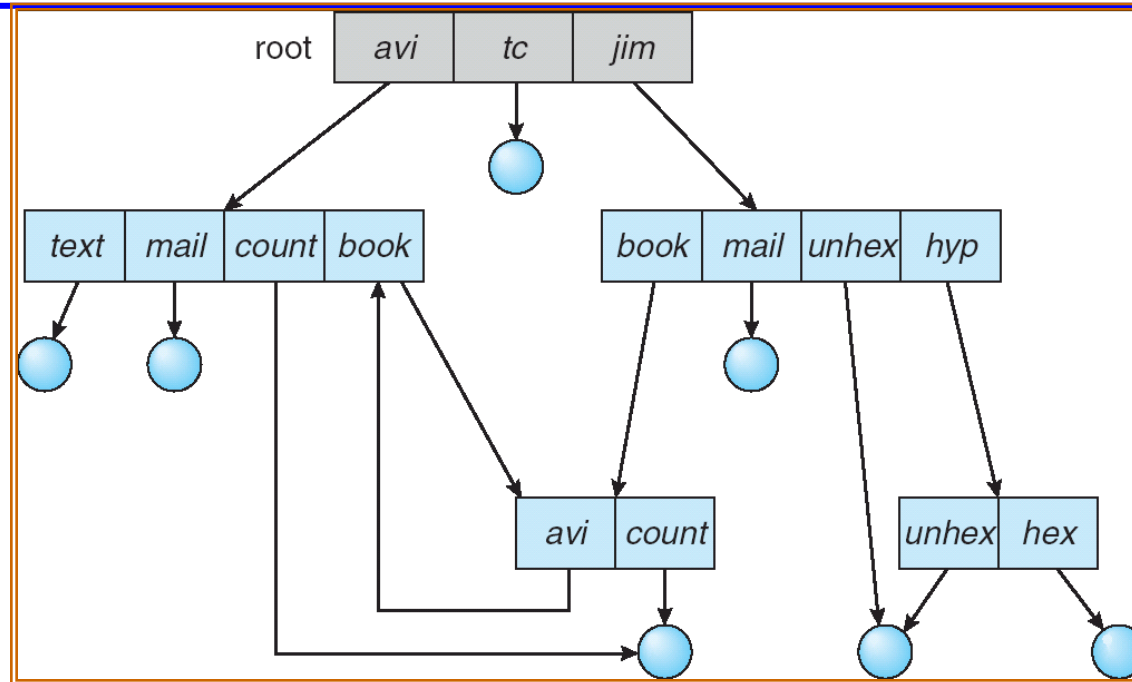
- Directories organized into a hierarchical structure
 - Seems standard, but in early 70's it wasn't
 - Permits much easier organization of data structures
- Entries in directory can be either files or directories
- Files named by ordered set
 - (e.g., `/classes/cs330/lecture01.pdf`)

Directory Structure



- Not really a hierarchy!
 - Many systems allow directory structure to be organized as an acyclic graph or even a (potentially) cyclic graph
 - Link: another name (pointer) to an existing file

Directory Structure

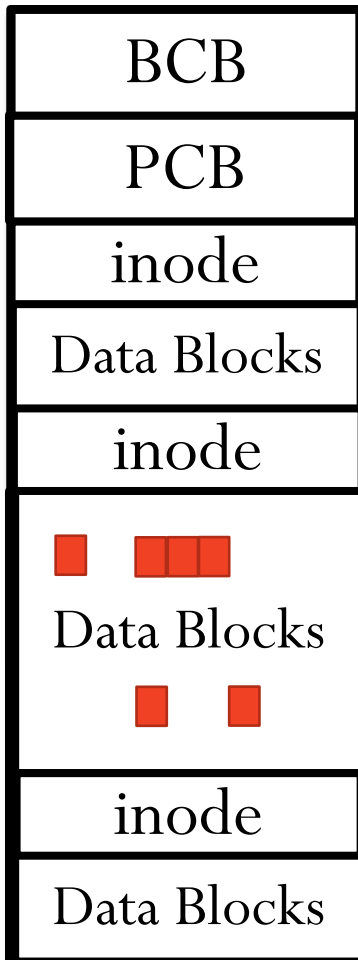


- **Name Resolution:** The process of converting a logical name into a physical resource (like a file)
 - Traverse succession of directories until reach target file
 - example: /avi/book/count

How?

Directory Structure (Con't)

- How many disk accesses to resolve “/classes/cs330/lecture01.pdf” (1MB)?

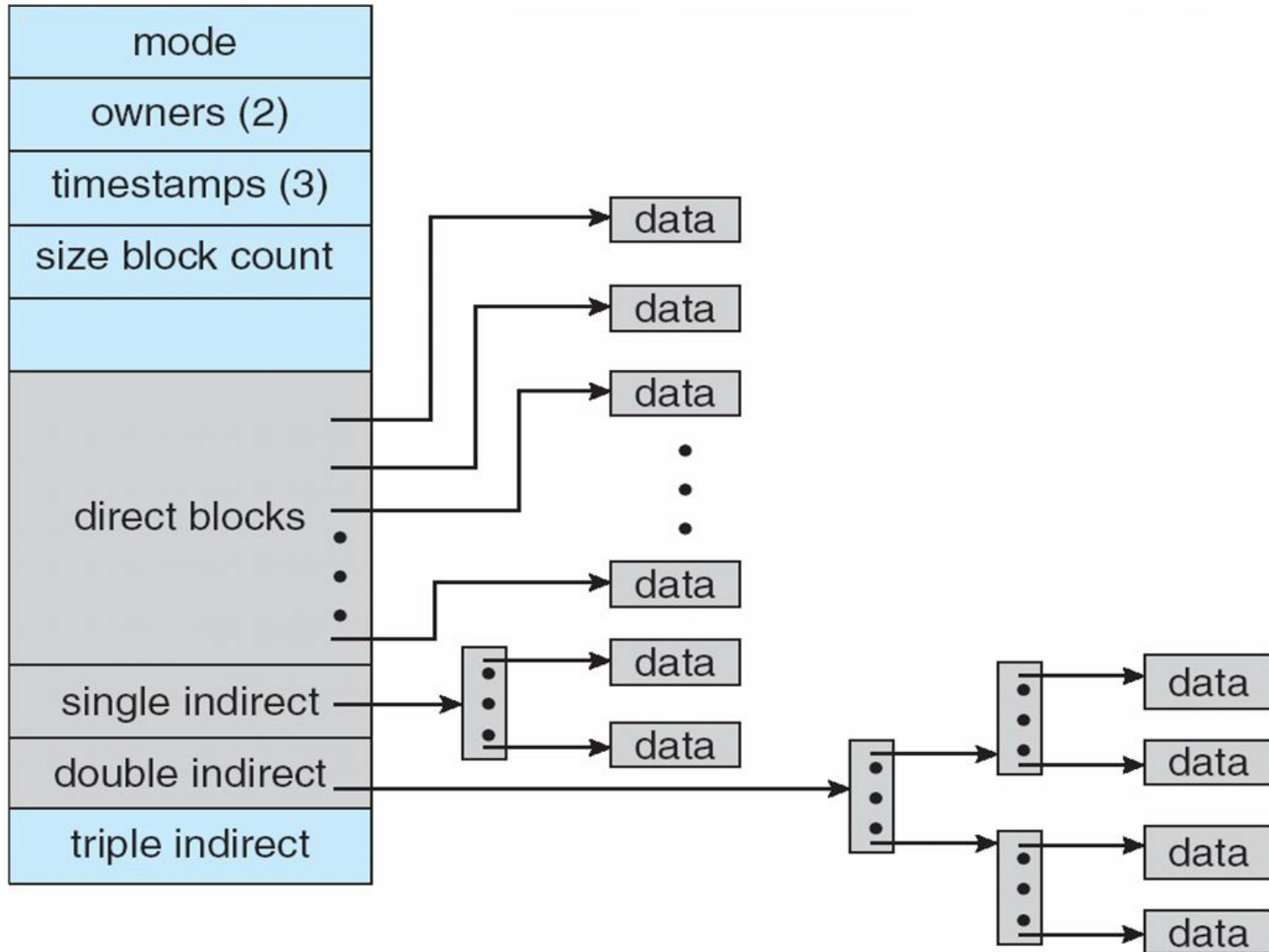


How can we reach the data blocks of “lecture01.pdf”?

Traverse the followings in order:

- “root”, “classes”, “cs330”, “lecture01.pdf”

Combined Scheme: UNIX UFS (4K bytes per block)



Naming

- How many disk accesses to resolve

“/users/i/insik.shin/thumbnail.jpg” (1KB)?

- 1) Read in i-node for root (fixed spot on disk)
- 2) Read in first data block for root
- 3) Read in i-node for “users”
- 4) Read in first data block for “users”; search for “i”
- 5) Read in i-node for “i”
- 6) Read in first data block for “i”; search for “insik.shin”
- 7) Read in i-node for “insik.shin”
- 8) Read in first data block for “insik.shin”; search for “thumbnail.jpg”
- 9) Read in i-node for “thumbnail.jpg”
- 10) Read in first data block for “thumbnail.jpg”

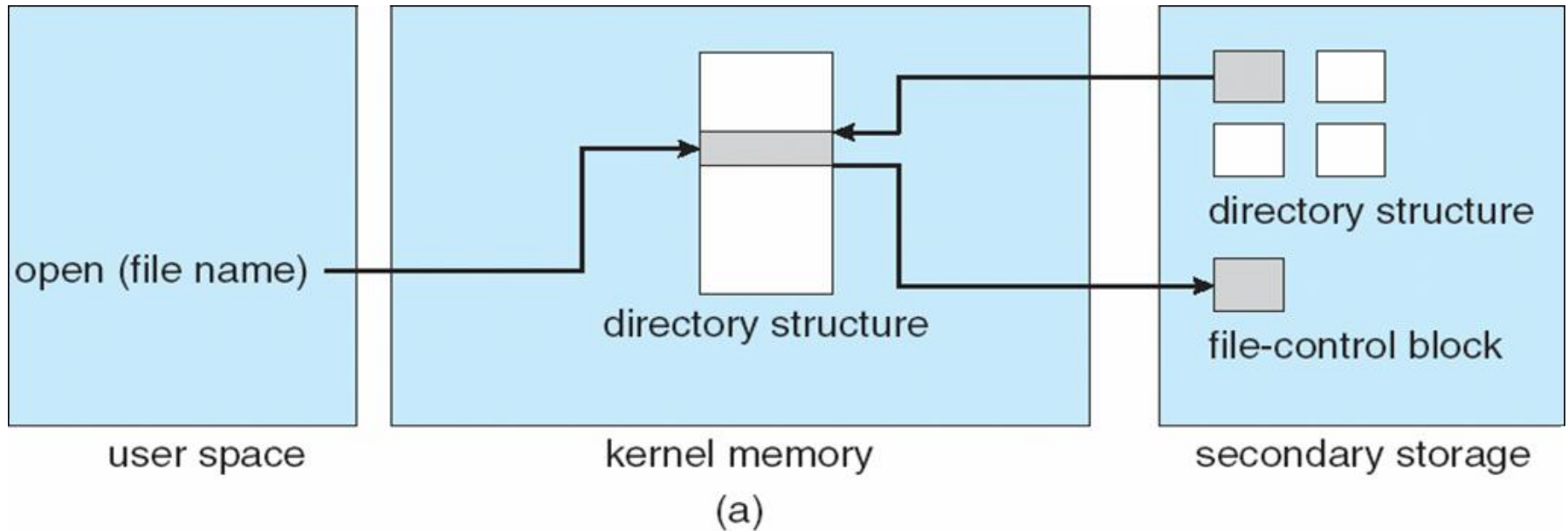
File Systems

- Issues on directory
 - Most file operations involve searching the directory for the entry associated with the file name
 - How to resolve this?

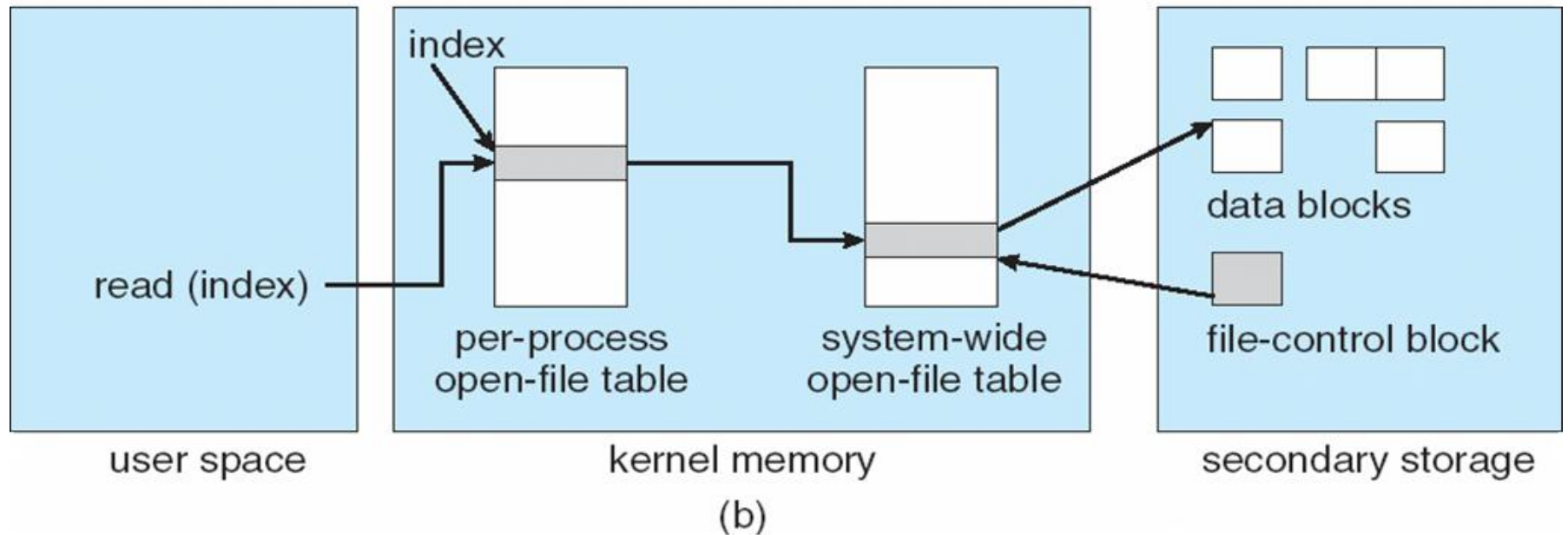
Open File

- Open-file table
 - To avoid constant searching, OS keeps a small table containing information about all open files (*open-file table*)
 - When a file operation is requested, the file is specified via an index into this table, so no searching is required
- File operations
 - $Open(F_i)$ – search the directory structure on disk for entry F_i , and move the content of entry to memory
 - $Close(F_i)$ – move the content of entry F_i in memory to directory structure on disk

In-Memory File System Structures



In-Memory File System Structures



Open Files

- Several pieces of data are needed to manage open files:
 - **File pointer**
 - pointer to last read/write location, per process that has the file open
 - **File-open count**
 - counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
 - **Disk location of the file**
 - cache of data access information
 - **Access rights**
 - per-process access mode information

Tree-Structured Directories (Cont)

- **Absolute** or **relative** path name
- Creating a new file/directory is done in current directory

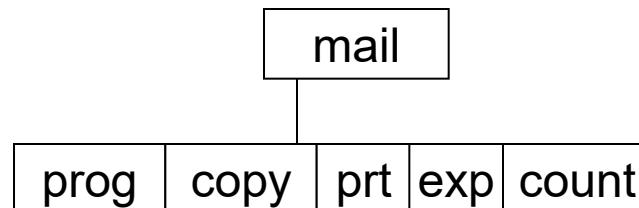
`mkdir <dir-name>`

- Delete a file

`rm <file-name>`

Example: if in current directory `/mail`

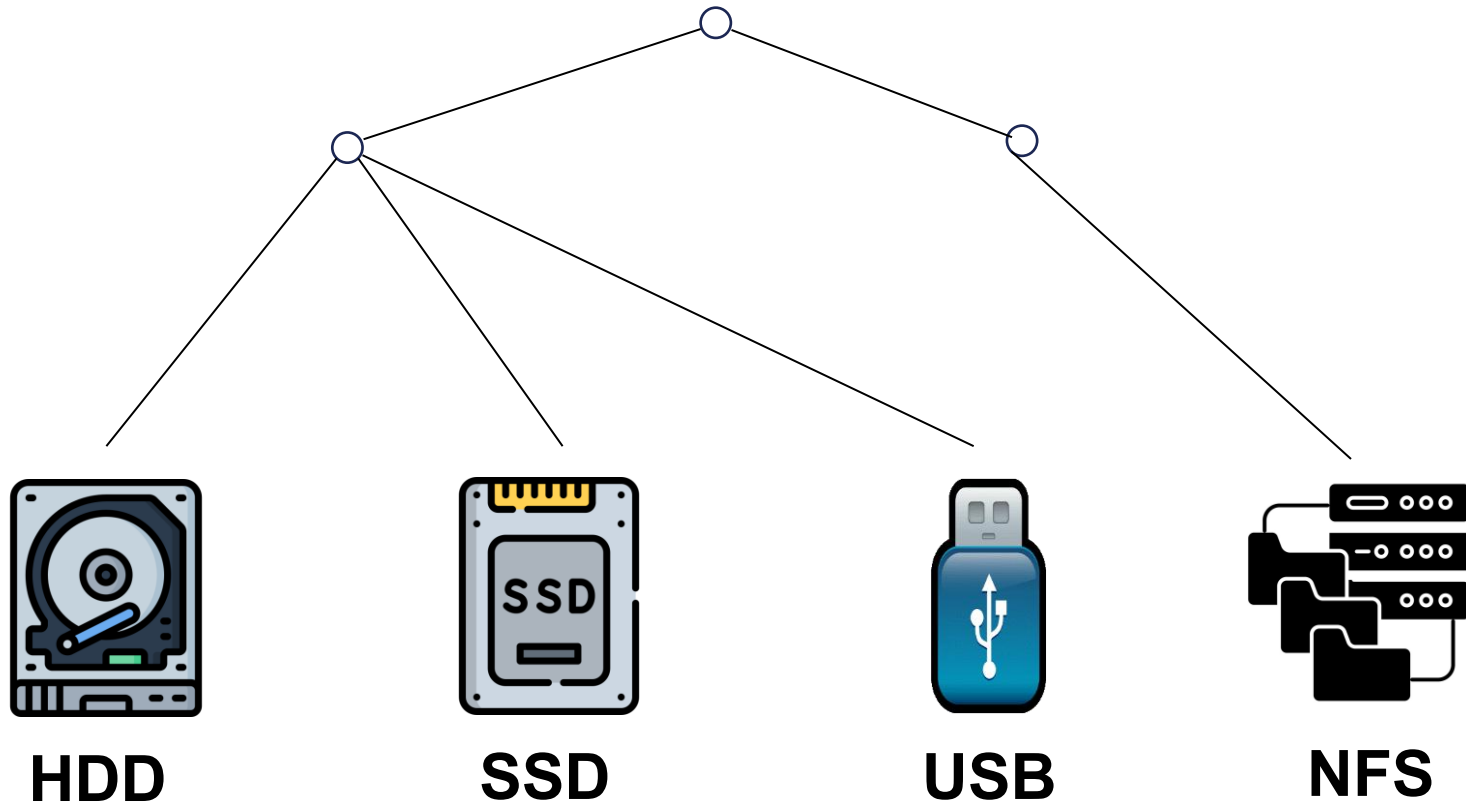
`mkdir count`



Deleting “mail” $\Rightarrow$ deleting the entire subtree rooted by “mail”

File System Mounting

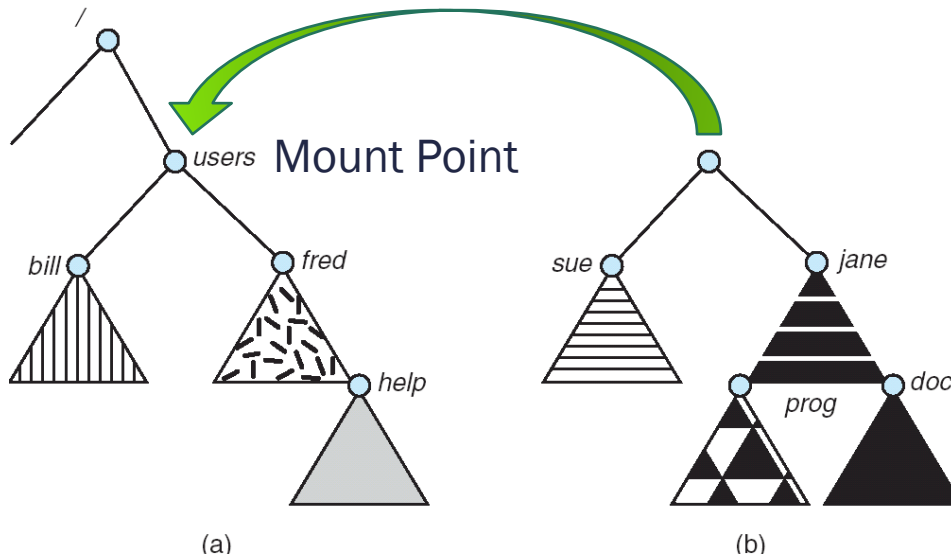
- Mounting
 - Files can be spread out over several devices
 - You may want build one single big tree to accommodate all of them, the file hierarchy, rooted at “/”



File System Mounting

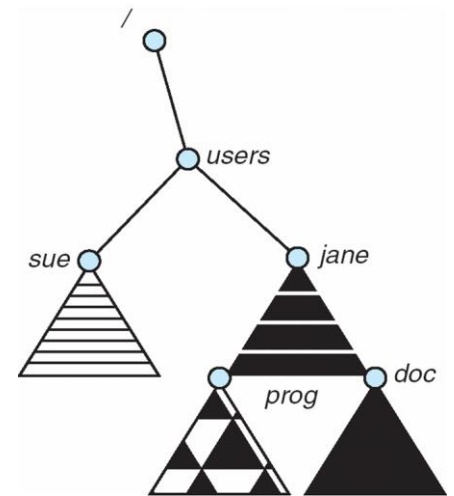
- Mounting
 - In Unix, it arranges all files accessible in one single big tree, the file hierarchy, rooted at “/”
 - Files can be spread out over several devices
 - A file system (from another device) must be **mounted** before it can be accessed
 - An unmounted file system is mounted at a **mount point**, which is typically an empty directory
- Example
 - The second partition of a hard disk is mounted with the command:
 - \$ **mount** /dev/hda2 /new/subdir
 - and unmounted with the command:
 - \$ **umount** /dev/hda2 or \$ **umount** /new/subdir

Mounting



(a) Existing File System

(b) Unmounted Partition



File System Mounting

- File system type
 - OS inspects the structures of a device & determines the type of file system
- A mounting point
 - Path is constructed from the mounting point
- Verification process
 - OS asks the device driver to read the device directory and verifies that the directory has the expected format

File System Mounting

- Mounting semantics
 - Systems impose semantics to clarify functionality
 - Mounting point
 - Only allows an empty directory
 - Mount a new FS and hide existing FS; restore existing one after terminating mounting
 - Mounting
 - Allow one mount per FS
 - Allow multiple mountings

File Sharing

- Sharing of files on multi-user systems is desirable
- Sharing may be done through a **protection** scheme
- On distributed systems, files may be shared across a network
- Network File System (NFS) is a common distributed file-sharing method

File Sharing – Multiple Users

- **User IDs** identify users, allowing permissions and protections to be per-user
- **Group IDs** allow users to be in groups, permitting group access rights

File Sharing – Remote File Systems

- Uses networking to allow file system access between systems
 - Manually via programs like FTP
 - Automatically, seamlessly using **distributed file systems**
 - Semi automatically via the **world wide web**
- **Client-server** model allows clients to mount remote file systems from servers
 - Server can serve multiple clients
 - **NFS** is standard UNIX client-server file sharing protocol
 - **CIFS** is standard Windows protocol
 - Standard operating system file calls are translated into remote calls

File Sharing – Consistency Semantics

- **Consistency semantics** specify how multiple users access a shared file simultaneously
 - How much difference allowed?
 - in some situation, different users may have different view on the shared file, then we may define the level of difference (e.g., Google docs, SNS)



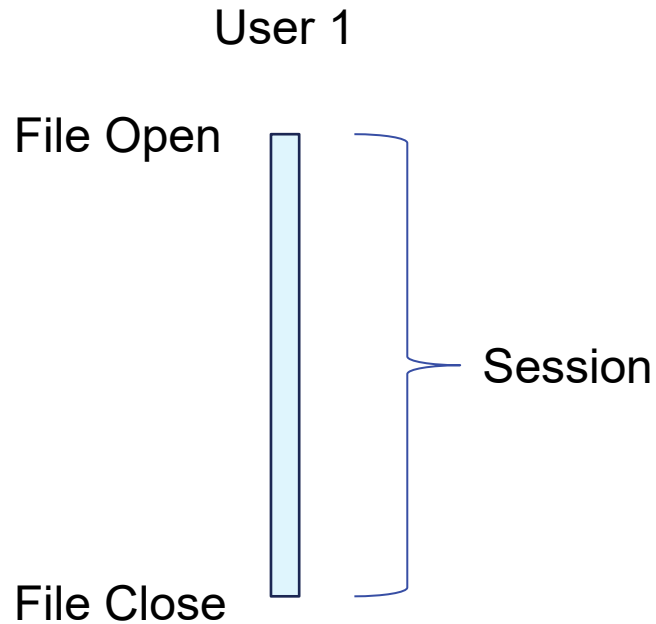
File Sharing – Consistency Semantics

- **Consistency semantics** specify how multiple users access a shared file simultaneously
 - How much difference allowed?
 - in some situation, different users may have different view on the shared file, then we may define the level of difference (e.g., Google docs, SNS)
 - Semantics for concurrent accesses:
 - What happens when one process reads while another writes?
 - What happens when two processes open the same file?

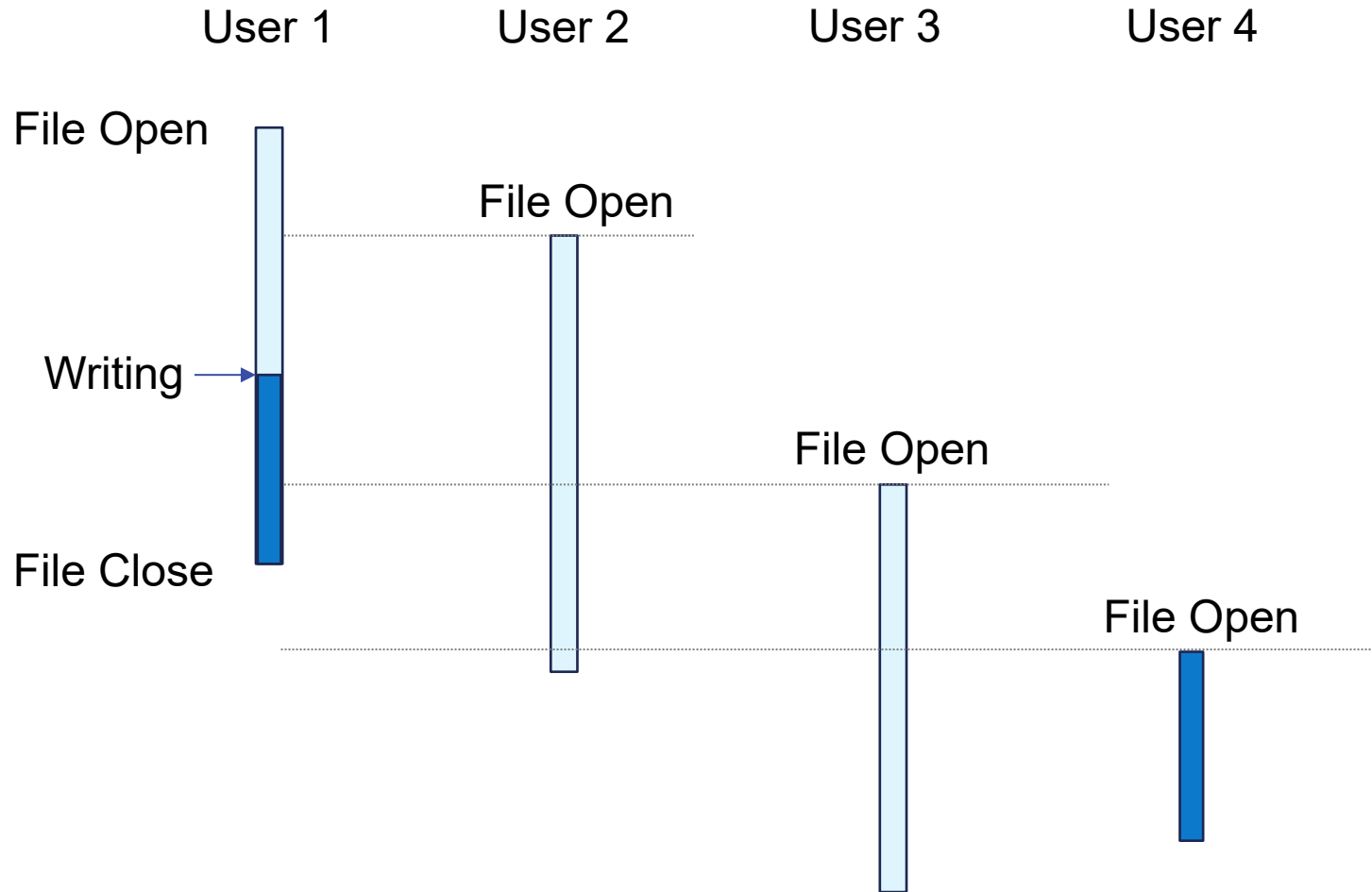
File Sharing – Consistency Semantics

- Unix file system (UFS) implements:
 - Writes to an open file visible *immediately* to other users of the same open file
 - Sharing file pointer to allow multiple users to read and write concurrently (i.e., *fork()*)
- AFS has session semantics (Andrew File System)
 - A session begins/ends when a file is open/closed
 - Writes become visible only to new sessions starting after the file has been closed

Andrew File System – Session Semantics



Andrew File System – Session Semantics



Protection

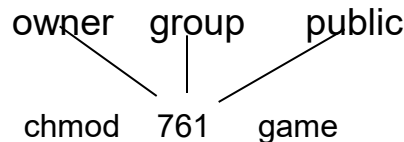
- File owner/creator should be able to control:
 - what can be done
 - by whom
- Types of access
 - **Read**
 - **Write**
 - **Execute**
 - **Append**
 - **Delete**
 - **List**

Access Lists and Groups

- Mode of access: read, write, execute
- Three classes of users

			RWX
a) owner access	7	⇒	1 1 1
			RWX
b) group access	6	⇒	1 1 0
			RWX
c) public access	1	⇒	0 0 1

- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a particular file (say *game*) or subdirectory, define an appropriate access.

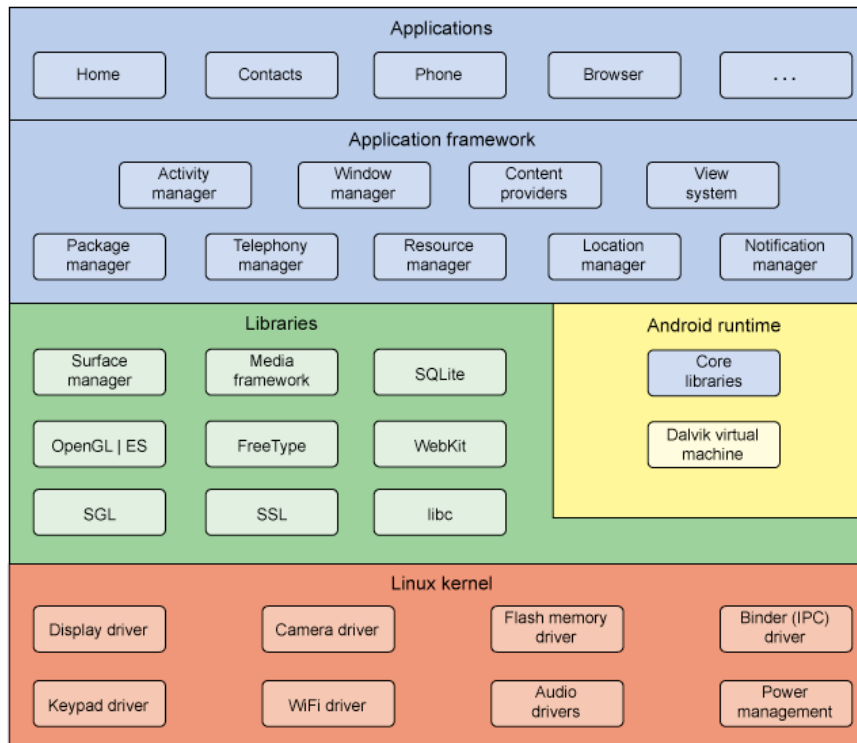


Attach a group to a file: chgrp G game

A Sample UNIX Directory Listing

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

Android Architecture



----- **Applications**

----- **Android Framework**

----- **Libraries**

----- **Linux Kernel**

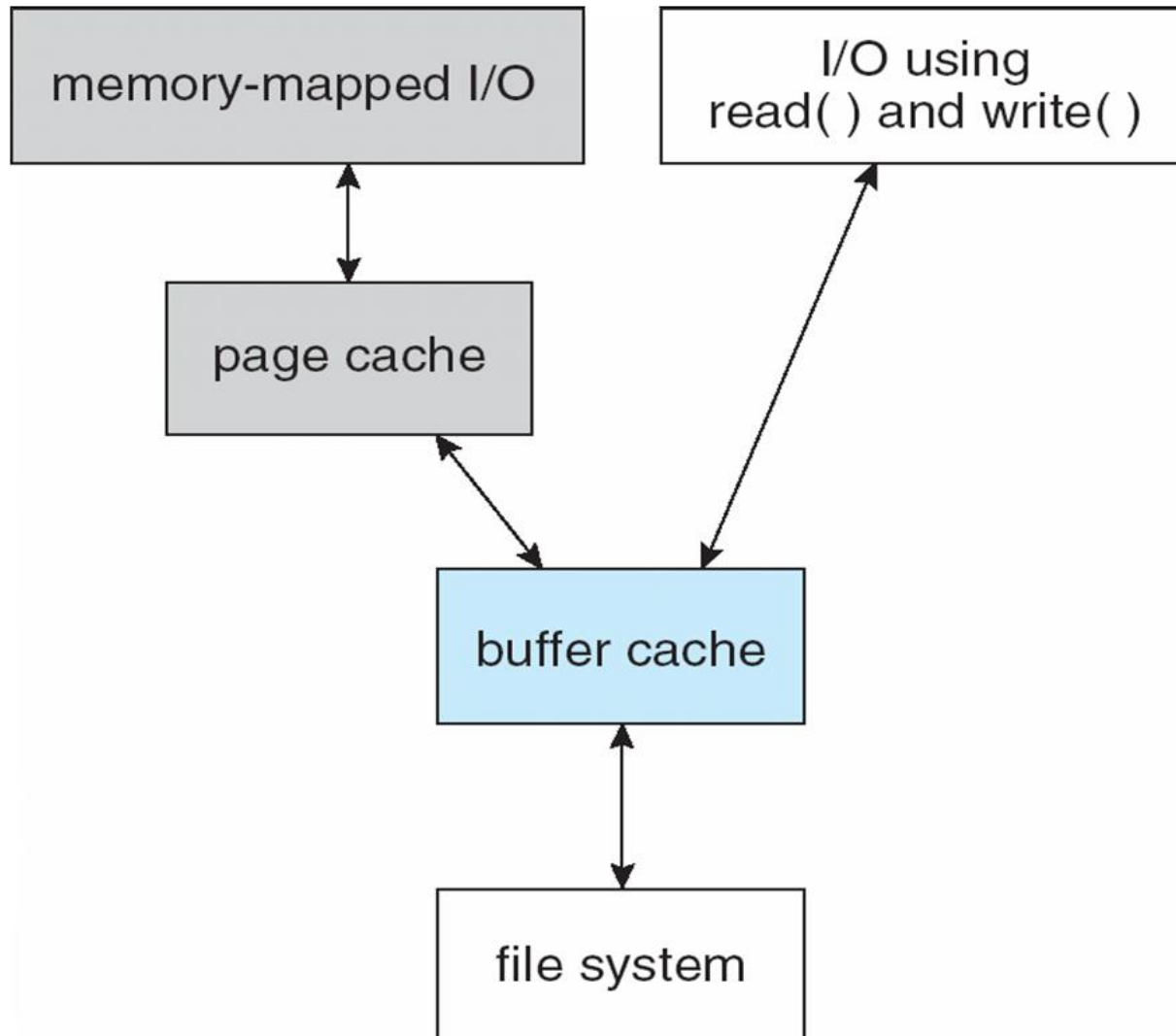
Performance

- Performance
 - Keeping data and metadata close together
 - **Buffer cache** - separate section of main memory for frequently used (disk) blocks
 - **Synchronous** writes sometimes requested by apps or needed by OS
 - No buffering / caching – writes must hit disk before acknowledgement
 - **Asynchronous** writes more common, buffer-able, faster
 - free-behind and read-ahead – techniques to optimize sequential access

Page Cache

- A **page cache** caches pages rather than disk blocks using virtual memory techniques
 - Memory-mapped I/O uses a page cache
 - Routine I/O through the file system uses the buffer (disk) cache
- Caching file data using virtual address is far more efficient than caching through physical disk blocks

I/O Without a Unified Buffer Cache

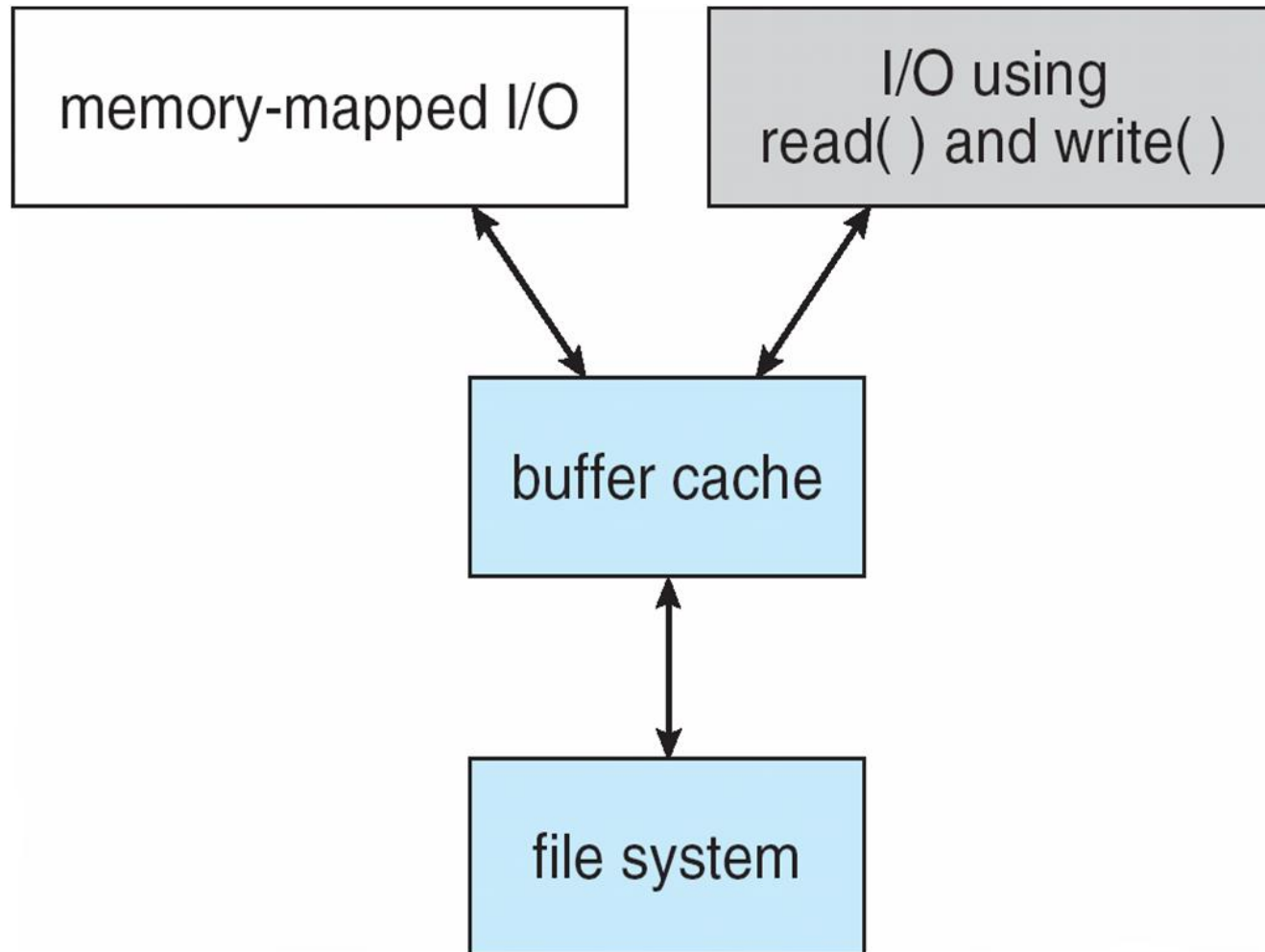


Unified Buffer Cache

- A **unified buffer cache** uses the same page cache to cache both memory-mapped pages and ordinary file system I/O to avoid **double caching**

But which caches get priority, and what replacement algorithms to use?

I/O Using a Unified Buffer Cache



Reliability

- File system consistency
 - File system can go into an inconsistent state if cached blocks are not written out due to the system crash
 - It is especially critical if it happens to directory / i-node / free-block-list blocks
 - Some utility programs for checking file system consistency
 - scandisk (Windows), fsck (UNIX)

Reliability – Consistency Checking

- fsck : checking blocks
 - Read all the i-nodes and mark used blocks
 - Examines the free list and mark free blocks

Consistent

	0	1	2	3	4	5	6	7
Blocks in use	1	1	0	1	0	1	1	1
Free blocks	0	0	1	0	1	0	0	0

Missing block

-- add it to the free list

	0	1	2	3	4	5	6	7
Blocks in use	1	1	0	1	0	1	1	1
Free blocks	0	0	0	0	1	0	0	0

Duplicated free block

-- rebuild the free list

	0	1	2	3	4	5	6	7
Blocks in use	1	1	0	1	0	1	1	1
Free blocks	0	0	1	0	2	0	0	0

Duplicated data block

-- allocate a new block and copy

	0	1	2	3	4	5	6	7
Blocks in use	1	1	0	1	0	2	1	1
Free blocks	0	0	1	0	1	0	0	0

Recovery

- Recovery from backup
 - Use system programs to **back up** data from disk to another storage device (magnetic tape, other magnetic disk, optical)
 - Recover lost file or disk by **restoring** data from backup
- Recovery from log

Log Structured File Systems

- Log structured (or journaling) file systems record each update to the file system as a transaction
- All transactions are written to a log
 - A transaction is considered committed once it is written to the log
 - Sometimes to a separate device or section of disk
 - However, the file system may not yet be updated
- The transactions in the log are asynchronously written to the file system
 - When the file system is modified, the transaction is removed from the log
- If the file system crashes, all remaining transactions in the log must still be performed

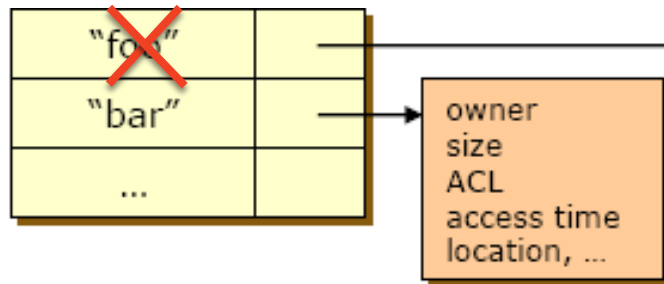
Log Structured File Systems

- For example, deleting a file on a Unix file system involves two steps:
 1. Removing its directory entry.
 2. Marking space for the file and its inode as free in the free space map.
- If a crash occurs between steps 1 and 2, there will be an orphaned inode and hence a storage leak.
- On the other hand, if only step 2 is performed first before the crash, the not-yet-deleted file will be marked free and possibly be overwritten by something else.

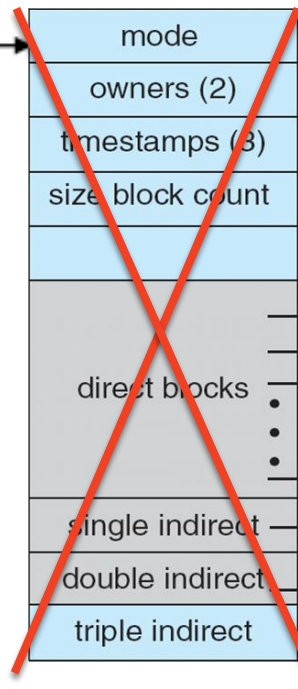
Deleting file "foo"

- ① Removing its directory entry.
- ② Marking space for the file and its inode as free in the free space map.

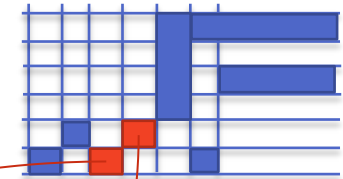
① Directory Table



FCB (i-node)



② Free Block List



Data Blocks

